EECS3311 Stage 1
Project Design Report

AI Used-Car Evaluation Assistant

EECS3311 Fall 2026

Yixing Yang

219908102

Contents

1. Project Overview

2. Detailed Feature Specifications

3. UML Class Diagram

4. Design Pattern Explanations

5. Use-Case Diagram

6. Detailed Use-Case Descriptions

7. Sequence Diagrams

8. Feature-to-Design Traceability Table

9. Feature Implementation Explanations

Diagram insertion guide

Section 3: Class Diagram. Section 5: Use-Case Diagram. Section 7: SD01–SD06, each at its labeled insertion location.

Task references retained in the source text correspond to Sections 1–2 (Task 1), Sections 3–7 (Task 2), Section 8 (Task 3), and Section 9 (Task 4).

1. Project Overview

Project Title

AI Used-Car Evaluation Assistant

Problem and Purpose

Buying a used vehicle requires a buyer to combine many different pieces of information, such as the vehicle's year, make, model, mileage, asking price, transmission, accident history, maintenance history, fuel consumption, known reliability concerns, and the buyer's own budget and priorities. For inexperienced buyers, these factors are difficult to evaluate consistently. Important information may also be missing from a seller's advertisement, and a buyer may focus on one attractive detail, such as a low price, while overlooking a high mileage value, unknown maintenance history, or a conflict with the buyer's stated requirements.

The proposed AI Used-Car Evaluation Assistant is an agent-based software system that helps a user organize, analyze, and compare used-vehicle information before deciding whether a vehicle is worth further consideration. The system is not intended to replace a professional pre-purchase inspection or guarantee the condition of a car. Instead, it provides a structured decision-support process. The software combines deterministic Java components for validation, calculations, preference checking, and local data retrieval with an LLM-based agent for natural-language understanding, explanation, clarification, and multi-factor recommendation.

Target Users

The primary target users are ordinary used-car buyers, especially first-time or relatively inexperienced buyers who want a structured way to review a vehicle listing. A user may know a budget and several preferences but may not know which information is missing, how different factors should be compared, or what questions should be asked before viewing the vehicle. The system is designed for a single end user rather than dealers or marketplace operators, which keeps the project scope focused and makes the later implementation and testing manageable.

What the Agent Can Do

The agent can accept structured vehicle information entered by the user or unstructured text copied from a used-car advertisement. It can extract listing details, detect missing or uncertain information, retrieve relevant notes from a small local vehicle knowledge base, invoke deterministic tools for cost estimation and constraint checking, compare two vehicles, generate a vehicle-specific inspection checklist, ask follow-up questions when critical information is missing, and produce a personalized evaluation report. The report explains the most important positive factors, risks, uncertainties, conflicts with the user's preferences, and an overall advisory recommendation.

The agent follows a multi-step process instead of forwarding every request directly to an LLM. Depending on the requested feature, it first validates the available data, decides whether more information is needed, selects and invokes appropriate local tools, retrieves relevant knowledge, maintains the user's preference and current-evaluation context, and only then uses the LLM where natural-language interpretation or reasoning is beneficial. This makes the AI component part of a larger software design rather than a simple chatbot.

Why an AI Agent Is Appropriate

A purely deterministic program is effective for precise calculations and rule checks, but it is less suitable for interpreting free-form seller descriptions, recognizing uncertainty expressed in natural language, deciding which follow-up question is most useful, and explaining trade-offs in a way that is understandable to a non-expert. An AI agent is therefore appropriate because the problem contains both structured tasks and judgment-oriented natural-language tasks. The agent can coordinate several tools, retain relevant context, reason over the combined results, and adapt the final explanation to the user's preferences.

At the same time, important calculations and constraints will not be delegated blindly to the LLM. Numerical cost estimates, required-field validation, local data lookup, and explicit user-constraint checks are handled by deterministic software components. This separation reduces hallucination risk and creates components that can be tested with conventional unit tests in Stage 3.

Planned AI/LLM Model

The planned primary LLM is Gemini, accessed through an API. The rest of the application will not communicate with the provider-specific API directly. Instead, it will use an LLMClient abstraction with a provider adapter. This keeps the application loosely coupled to one vendor and allows a different course-approved LLM provider to be substituted later without changing the vehicle domain logic, cost calculations, local repository, or user-interface code.

How the AI Model Interacts with the Rest of the Software

The GUI and CLI send user requests to a common application layer rather than calling the LLM directly. A central UsedCarAgent coordinates the evaluation workflow. The agent can invoke deterministic tools such as a VehicleKnowledgeTool, CostCalculatorTool, and ConstraintCheckerTool through a ToolManager. The VehicleKnowledgeTool reads a small local JSON/CSV knowledge base containing curated reliability notes and common inspection concerns. The agent then prepares only the relevant structured context for the LLM. The LLM is used for tasks such as listing-text interpretation, clarification-question generation, risk explanation, inspection-checklist generation, and synthesis of the final recommendation. LLM responses are converted back into structured application results before being displayed to the user.

The intended high-level workflow is: User Request → GUI/CLI → Application Facade → UsedCarAgent → Validation and Tool Selection → Local Tools/Repository → LLMClient when needed → Structured Result → GUI/CLI. This architecture makes tool use, retrieval, memory, decision making, and multi-step execution visible in the software design.

GUI and CLI Access

The graphical user interface will provide the main interaction method. It will contain a vehicle-input area, a listing-text analyzer, a buyer-preference panel, comparison controls, an evaluation-results area, and access to inspection checklists and reports. The command-line interface will expose the same major application services through commands such as evaluate, analyze-listing, compare, estimate-cost, check-missing, checklist, and report. Both interfaces will reuse the same underlying application commands and domain services so that the project does not duplicate business logic.

Planned Design Pattern Support and Scope Control

The later UML design will use at least five course design patterns in meaningful roles: Strategy for interchangeable evaluation priorities, Adapter for the external LLM provider, Factory Method for creating the appropriate evaluation strategy or analysis component, Command for exposing the same operations through both GUI and CLI, and Facade for providing a simple entry point to the agent and its supporting subsystems. The detailed participants and rationale for these patterns will be provided in Task 2.

To keep Stage 2 and Stage 3 practical, the project deliberately excludes real-time marketplace scraping, Carfax integration, live insurance quotes, map services, payment functions, and a large production database. Vehicle knowledge is stored locally, only one LLM service is required, and deterministic calculations are kept simple and independently testable. This limited scope still provides meaningful agent behavior while avoiding unnecessary external dependencies.

2. Detailed Feature Specifications

The project contains twelve distinct and non-trivial features. Each feature is classified as deterministic, AI-based, or hybrid, and each one is designed so that it can later be traced to use cases, classes, methods, sequence diagrams, implementation, and tests.

F01 — Vehicle Information Entry and Management

1. **Feature ID and Name —** F01 — Vehicle Information Entry and Management
2. **Description —** Allows the user to create and edit a structured vehicle record containing the information needed for later evaluation. The feature standardizes data before any AI reasoning occurs and gives the rest of the system a consistent Vehicle object to work with.
3. **User Interaction —** In the GUI, the user opens the Vehicle Input panel and enters fields such as year, make, model, trim (optional), mileage, asking price, transmission, fuel type, accident-history status, maintenance-history status, seller notes, and optional fuel-consumption data. The user can review and edit the record before evaluation. The CLI provides an equivalent vehicle-add or vehicle-edit command with named arguments or prompts.
4. **Input —** Vehicle year, make, model, optional trim, mileage, asking price, transmission type, fuel type, accident-history status, maintenance-history status, optional fuel-consumption value, and free-text notes.
5. **Output —** A validated Vehicle record that can be stored in the current evaluation session and used by all later features. The GUI displays a concise vehicle summary and identifies any basic validation problems.
6. **AI Involvement —** Deterministic. The feature uses normal Java validation and object creation; no LLM call is required.
7. **Expected Workflow —** The system receives the user's field values, validates numeric ranges and required fields, normalizes simple values such as transmission labels, creates or updates a Vehicle object, stores it in the current session, and displays the normalized record. Other features can then reuse the same object instead of reparsing the data.
8. **Error/Alternative Cases —** If a required field is empty, a numeric field is malformed, mileage or price is negative, or the year is outside a reasonable range, the system rejects the invalid value and explains which field must be corrected. Optional fields may remain unknown; they are recorded explicitly as unknown rather than silently guessed.

F02 — Used-Car Listing Text Analysis

1. **Feature ID and Name —** F02 — Used-Car Listing Text Analysis
2. **Description —** Allows the user to paste an unstructured seller advertisement and uses the LLM to identify useful vehicle information. The feature reduces manual data entry while keeping the user in control of the extracted values.
3. **User Interaction —** In the GUI, the user pastes advertisement text into the Listing Analyzer panel and selects Analyze Listing. The system displays the extracted fields beside the original text and asks the user to confirm or edit them before they are saved. The CLI provides an analyze-listing command that accepts pasted text or a local text file.
4. **Input —** Free-form listing text that may contain a mixture of vehicle specifications, seller claims, maintenance information, accident statements, pricing information, and irrelevant promotional language.
5. **Output —** A structured draft containing any detected year, make, model, mileage, price, transmission, accident information, maintenance information, and other useful notes. Values that are unclear are marked uncertain instead of being presented as confirmed facts.

6. **AI Involvement —** AI-based with deterministic validation. The LLM performs natural-language extraction, while normal Java code validates the returned structure and data types.
7. **Expected Workflow —** The application sends the listing text and a constrained extraction request to the LLMClient. The LLM returns structured candidate fields. A response parser validates the format, and the GUI shows the result to the user. Only confirmed or user-approved values are copied into the Vehicle record.
8. **Error/Alternative Cases —** If the LLM response is malformed, the system reports the parsing problem and allows a retry. If important information is not present in the advertisement, the field remains unknown. The system must not invent a VIN, accident history, service record, or specification that is not supported by the supplied text.

F03 — Buyer Preference Profile

1. **Feature ID and Name —** F03 — Buyer Preference Profile
2. **Description —** Stores the user's major buying constraints and priorities so later evaluations are personalized rather than generic. The profile also provides simple memory across evaluations.
3. **User Interaction —** In the GUI, the user opens the Preferences panel and sets a maximum budget, maximum preferred mileage, acceptable transmission types, preferred body type, fuel-economy importance, reliability importance, and tolerance for accident history. The CLI provides a profile-set command and a profile-show command.
4. **Input —** Maximum budget, optional maximum mileage, acceptable transmission type(s), optional body-style preference, relative importance of reliability and fuel economy, and accident-history tolerance.
5. **Output —** A UserPreference profile stored locally and available to the evaluation, comparison, conflict-detection, and recommendation features.
6. **AI Involvement —** Deterministic. The feature stores and retrieves structured preference values without using the LLM.
7. **Expected Workflow —** The user enters or updates preferences, the system validates the values, saves them to the current profile, and makes the profile available to the UsedCarAgent. When another feature runs, the agent retrieves the profile and uses it as explicit constraints rather than relying on assumptions.
8. **Error/Alternative Cases —** If the user enters contradictory or invalid preference values, such as a negative budget, the system requests correction. If some preferences are not specified, the system treats them as neutral rather than inventing a preference.

F04 — Missing Information and Data-Quality Detection

1. **Feature ID and Name —** F04 — Missing Information and Data-Quality Detection
2. **Description —** Checks whether the available vehicle information is sufficient for a meaningful evaluation and identifies missing, uncertain, or internally inconsistent data before the agent produces a recommendation.
3. **User Interaction —** The GUI shows a Missing Information section whenever a vehicle is analyzed or evaluated. The user can also run the check directly. The CLI provides a check-missing command.
4. **Input —** The current Vehicle record, any extracted listing data, and the current buyer profile.
5. **Output —** A prioritized list of missing or questionable items, such as unknown accident history, unknown maintenance history, missing asking price, unclear mileage, or conflicting values between manually entered data and listing text.
6. **AI Involvement —** Hybrid. Deterministic rules check required fields and direct conflicts, while the LLM may interpret ambiguous seller wording and help explain why a missing item matters.
7. **Expected Workflow —** The system first runs deterministic field and consistency checks. If free-text information is ambiguous, the agent can ask the LLM to classify the statement as known, unknown, or

uncertain. The feature then produces a structured MissingInformationResult that can trigger clarification questions before final evaluation.

8. **Error/Alternative Cases —** If there is too little information to proceed, the system does not produce an overconfident recommendation; it explains that the evaluation is incomplete. If two sources disagree, both values are shown and the user is asked which one should be trusted.

F05 — Local Vehicle Knowledge Retrieval

1. **Feature ID and Name —** F05 — Local Vehicle Knowledge Retrieval
2. **Description —** Retrieves relevant reliability and inspection notes from a small local vehicle knowledge base. This gives the agent a concrete source of domain information without requiring a live external vehicle-data API.
3. **User Interaction —** The feature normally runs automatically during evaluation. The GUI also allows the user to view the retrieved knowledge notes for the selected vehicle. The CLI provides a knowledge command for the current vehicle.
4. **Input —** Vehicle make, model, year or year range, and optionally engine or transmission information when available.
5. **Output —** A VehicleKnowledge result containing locally stored notes such as common areas to inspect, general reliability concerns, and source/category metadata used by the project dataset.
6. **AI Involvement —** Deterministic retrieval. The agent decides when to use the tool, but the lookup itself is performed by normal Java code against local JSON/CSV data.
7. **Expected Workflow —** The UsedCarAgent asks the ToolManager for the VehicleKnowledgeTool. The tool searches the local repository using normalized vehicle identifiers and returns the best matching notes. These notes become evidence available to risk assessment, comparison, checklist generation, and final recommendation.
8. **Error/Alternative Cases —** If the vehicle is not represented in the local knowledge base, the system returns a clear 'no local data available' result. The LLM is not allowed to pretend that missing local reliability data was retrieved. The evaluation may continue using the information that is actually available, with increased uncertainty.

F06 — Ownership Cost Estimation

1. **Feature ID and Name —** F06 — Ownership Cost Estimation
2. **Description —** Calculates simple, transparent ownership-cost estimates using user-provided numeric assumptions. The feature gives the project a meaningful deterministic calculation component that can later be unit tested.
3. **User Interaction —** In the GUI, the user opens the Cost Estimate panel and enters annual driving distance, estimated fuel consumption, fuel price, optional tax rate, and optional annual maintenance or insurance allowance. The CLI provides an estimate-cost command.
4. **Input —** Asking price, annual distance, fuel consumption in L/100 km, fuel price per litre, optional tax rate, optional annual maintenance allowance, and optional annual insurance estimate.
5. **Output —** A structured CostEstimate containing estimated annual fuel use, annual fuel cost, optional purchase tax, and an optional simple first-year total based only on the values the user supplied.
6. **AI Involvement —** Deterministic. All calculations use explicit formulas implemented in Java.
7. **Expected Workflow —** The system validates numeric inputs, applies the documented formulas, creates a CostEstimate object, and displays the breakdown. The result is also available to comparison and recommendation features. The calculation logic remains independent from the LLM so identical inputs always produce identical outputs.

8. **Error/Alternative Cases —** If a required numeric assumption is missing, the system either omits that portion of the estimate or asks the user to provide it. Invalid negative values are rejected. The feature clearly labels the output as an estimate and does not invent insurance, maintenance, or fuel-price values that were not provided.

F07 — Buyer Constraint Conflict Detection

1. **Feature ID and Name —** F07 — Buyer Constraint Conflict Detection
2. **Description —** Automatically checks whether a vehicle conflicts with explicit buyer constraints. This prevents the final AI recommendation from ignoring hard requirements such as budget, mileage, or transmission preference.
3. **User Interaction —** Conflict checks run automatically when the user evaluates or compares a vehicle. The GUI displays conflicts in a dedicated section, and the CLI includes a check-constraints command.
4. **Input —** The current Vehicle record and UserPreference profile.
5. **Output —** A list of ConstraintViolation objects describing each detected conflict, such as price above budget, mileage above the preferred maximum, unacceptable transmission type, or accident-history status outside the user's tolerance.
6. **AI Involvement —** Deterministic. The feature compares structured values using explicit rules.
7. **Expected Workflow —** The ConstraintCheckerTool receives the Vehicle and UserPreference objects, evaluates each applicable rule, records any violations and their severity, and returns the result to the agent. The agent must include these conflicts in later risk explanations and recommendations.
8. **Error/Alternative Cases —** If a preference needed for a check has not been specified, that check is skipped rather than treated as a failure. If the vehicle value is unknown, the result is marked unknown and may trigger clarification rather than a false pass.

F08 — AI-Assisted Vehicle Risk Assessment

1. **Feature ID and Name —** F08 — AI-Assisted Vehicle Risk Assessment
2. **Description —** Combines structured vehicle data, missing-information results, local knowledge, cost information, and detected constraint conflicts to produce an explainable risk assessment.
3. **User Interaction —** The user selects Evaluate Vehicle in the GUI after entering or confirming the vehicle information. The results panel shows a risk level, the main reasons, supporting tool results, and remaining uncertainties. The CLI provides an evaluate command.
4. **Input —** Vehicle record, MissingInformationResult, retrieved VehicleKnowledge, optional CostEstimate, ConstraintViolation list, and the UserPreference profile.
5. **Output —** A RiskAssessment with an advisory risk level such as Low, Moderate, High, or Insufficient Information, plus an explanation of the most important positive factors, risks, and uncertainties.
6. **AI Involvement —** Hybrid. Deterministic tools generate factual inputs and rule results; the LLM synthesizes these inputs into a coherent explanation and reasoned risk assessment.
7. **Expected Workflow —** The UsedCarAgent verifies that minimum information is available, invokes the required local tools through ToolManager, assembles a structured evidence package, and sends only that evidence and the evaluation instructions to the LLMClient. The returned assessment is parsed and checked for required fields before display.
8. **Error/Alternative Cases —** If critical information is missing, the agent may return Insufficient Information and invoke the clarification feature instead of forcing a Low/Moderate/High result. If the LLM produces a recommendation unsupported by the tool evidence, the system can reject the response and retry with stricter instructions.

F09 — Two-Vehicle Comparison

1. **Feature ID and Name —** F09 — Two-Vehicle Comparison
2. **Description —** Compares two candidate vehicles using the same structured criteria and user preferences. The feature helps a buyer understand trade-offs rather than evaluating each vehicle in isolation.
3. **User Interaction —** In the GUI, the user selects two saved/current Vehicle records and chooses Compare. A side-by-side result shows price, mileage, constraints, cost estimates, local knowledge availability, and AI-generated trade-off explanation. The CLI provides a compare command with two vehicle identifiers.
4. **Input —** Two Vehicle records, the UserPreference profile, optional CostEstimate objects, retrieved local knowledge, and constraint results for each vehicle.
5. **Output —** A VehicleComparison result showing deterministic side-by-side metrics and a concise AI explanation of which vehicle better matches the user's stated priorities and why.
6. **AI Involvement —** Hybrid. Numeric and rule-based comparisons are deterministic; the LLM explains the trade-offs using the structured comparison result.
7. **Expected Workflow —** The system validates that two different vehicles are available, computes comparable metrics for both, checks user constraints, retrieves local knowledge where available, creates a structured comparison package, and asks the LLM to explain the differences without changing the underlying calculated values.
8. **Error/Alternative Cases —** If important data is missing for one vehicle, the comparison marks that metric as unavailable and explains that the comparison is incomplete. The system does not treat missing information as automatically better or worse.

F10 — Vehicle-Specific Inspection Checklist Generation

1. **Feature ID and Name —** F10 — Vehicle-Specific Inspection Checklist Generation
2. **Description —** Generates a practical pre-purchase inspection checklist tailored to the selected vehicle and the information already known about it. The checklist converts the analysis into concrete questions and inspection points for the user.
3. **User Interaction —** The user selects Generate Inspection Checklist in the GUI after loading a vehicle. The checklist appears in grouped sections such as documents/history, exterior, interior, engine/drivetrain, road test, and questions for the seller. The CLI provides a checklist command.
4. **Input —** Vehicle record, local VehicleKnowledge notes, MissingInformationResult, and any known accident or maintenance information.
5. **Output —** A structured InspectionChecklist containing prioritized inspection items and seller questions relevant to the current vehicle.
6. **AI Involvement —** Hybrid. Local data and missing-information results provide factual context, while the LLM organizes and explains a vehicle-specific checklist.
7. **Expected Workflow —** The agent retrieves relevant vehicle knowledge, adds missing-information items, constructs a constrained checklist request, invokes the LLMClient, validates the returned checklist structure, and displays the checklist. The user can mark items as completed locally without requiring additional AI calls.
8. **Error/Alternative Cases —** If local knowledge is unavailable, the system generates a more general checklist and clearly states that model-specific notes were not found. If the LLM call fails, the system can still display deterministic missing-information questions and a basic generic checklist.

F11 — Clarification Question and Follow-Up Management

1. **Feature ID and Name —** F11 — Clarification Question and Follow-Up Management

2. **Description —** Allows the agent to pause an evaluation and ask targeted follow-up questions when important information is missing or ambiguous. This provides visible multi-step agent behavior and lightweight conversational memory.
3. **User Interaction —** When the system detects important gaps, the GUI displays one or more clarification questions in the evaluation panel. The user answers them and selects Continue Evaluation. The CLI presents the same questions sequentially and records the answers.
4. **Input —** MissingInformationResult, ambiguous listing statements, current Vehicle data, UserPreference profile, and the current evaluation-session history.
5. **Output —** A set of prioritized clarification questions and updated structured vehicle/session data after the user responds.
6. **AI Involvement —** AI-based with deterministic storage and validation. The LLM selects and phrases useful follow-up questions, while Java code records the answers and updates the corresponding fields.
7. **Expected Workflow —** The agent examines the missing-information result, asks the LLM for a small number of high-value questions, presents them to the user, validates the answers, updates the evaluation context, and resumes the interrupted workflow. The session retains the answers so the user is not repeatedly asked for the same information.
8. **Error/Alternative Cases —** The user may skip a question or answer 'unknown'. The system records that uncertainty and continues when possible. If the user provides an answer that conflicts with existing data, the system highlights the conflict and asks which value should be retained.

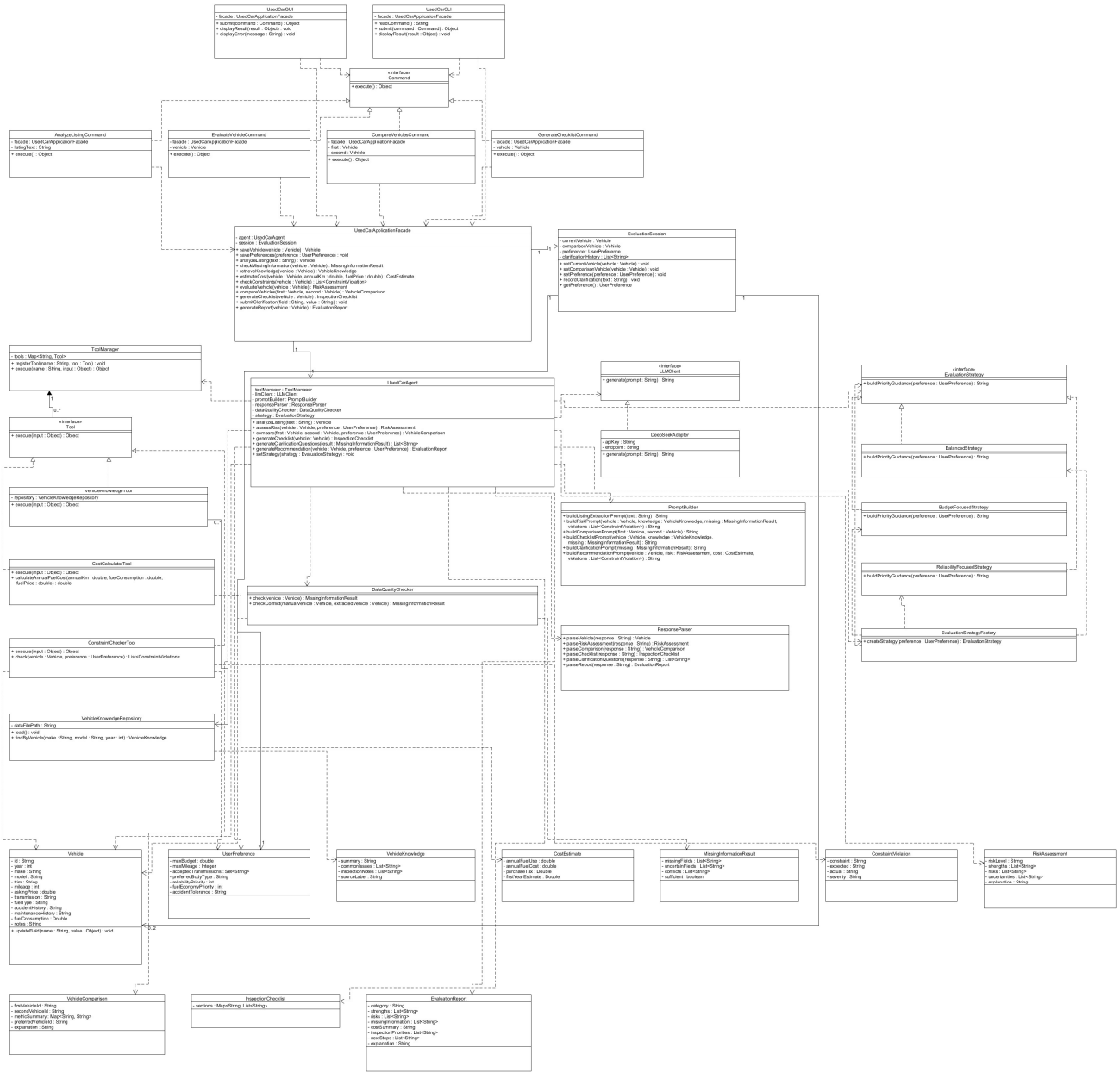
F12 — Personalized Purchase Recommendation and Evaluation Report

1. **Feature ID and Name —** F12 — Personalized Purchase Recommendation and Evaluation Report
2. **Description —** Produces the final integrated evaluation after the system has gathered available information and executed the relevant tools. The report is advisory and explains both the recommendation and the evidence behind it.
3. **User Interaction —** The user selects Generate Recommendation / Report in the GUI. The report view shows an overall advisory category, key strengths, key risks, missing information, constraint conflicts, cost summary, inspection priorities, and next-step suggestions. The CLI provides a report command that prints the same structured result in text form.
4. **Input —** Vehicle record, UserPreference profile, RiskAssessment, CostEstimate when available, ConstraintViolation list, local knowledge results, MissingInformationResult, and the current evaluation-session context.
5. **Output —** An EvaluationReport with an advisory category such as Strong Candidate, Consider with Caution, High Risk, or Insufficient Information, accompanied by concise reasons and next actions. The result can be saved locally for later review.
6. **AI Involvement —** Hybrid. Deterministic components supply the facts, calculations, constraints, and retrieved evidence; the LLM performs the final explanation and prioritization. The application validates the returned report structure before presenting it.
7. **Expected Workflow —** The UsedCarAgent checks whether an up-to-date risk assessment exists, invokes any missing tools, resolves or records remaining uncertainty, assembles a final evidence package, applies the selected EvaluationStrategy, and asks the LLM to generate the explanation. The application then creates the EvaluationReport object, stores it locally, and displays it through either GUI or CLI.
8. **Error/Alternative Cases —** If evidence is insufficient, the report uses Insufficient Information instead of presenting a confident recommendation. If a required tool fails, the report identifies which information could not be obtained. If the LLM service is unavailable, the deterministic results remain viewable and the user is told that the narrative recommendation could not be generated.

3. UML Class Diagram

This Task 2 design is based directly on the twelve features defined in Task 1. The UML diagrams will be created in UMLet and inserted at the marked locations. The textual design below defines the classes, interfaces, responsibilities, design-pattern participants, use cases, and sequence-diagram interactions so that all diagrams can remain consistent with each other and with the later Stage 2 implementation.

The class diagram should represent the major static structure of the AI Used-Car Evaluation Assistant. It should include the main presentation components, application facade, agent components, tool components, LLM integration, strategy classes, command classes, repositories, and domain/result objects. The final UMLet class diagram should show important attributes and methods together with associations, dependencies, realization/inheritance relationships, aggregation/composition where appropriate, and meaningful multiplicities.



3.1 Design Principles Applied

The design separates responsibilities so that the LLM does not become the entire application. Vehicle data, preferences, validation, cost calculations, local retrieval, API communication, prompt construction, response parsing, and user-interface concerns are assigned to separate components. This supports high cohesion because each class has a focused responsibility, and low coupling because components communicate through small interfaces such as LLMClient, Tool, Command, and EvaluationStrategy.

Encapsulation is used for domain state such as Vehicle and UserPreference. Dependency inversion is applied by making UsedCarAgent depend on the LLMClient abstraction rather than directly on Gemini-specific API code. Polymorphism is used through Tool, Command, and EvaluationStrategy interfaces. The design also avoids a single large 'god class': UsedCarAgent coordinates an evaluation, but specialized calculations, retrieval, data-quality checks, and LLM communication remain delegated to dedicated collaborators.

3.2 Major Classes and Interfaces

Presentation / Boundary Components

UsedCarGUI — Graphical boundary through which the user enters vehicles, pastes listing text, manages preferences, starts evaluations/comparisons, answers clarification questions, and views results.

Important Attributes: - facade : UsedCarApplicationFacade

Important Methods: + submit(command : Command) : Object; + displayResult(result : Object) : void; + displayError(message : String) : void

UsedCarCLI — Command-line boundary that provides access to the same major application operations without duplicating business logic.

Important Attributes: - facade : UsedCarApplicationFacade

Important Methods: + readCommand() : String; + submit(command : Command) : Object; + displayResult(result : Object) : void

Application / Coordination Components

UsedCarApplicationFacade — Single application entry point used by GUI and CLI. It hides the internal complexity of the agent, session, tools, and LLM-facing workflow.

Important Attributes: - agent : UsedCarAgent; - session : EvaluationSession

Important Methods: + saveVehicle(vehicle : Vehicle) : Vehicle; + savePreferences(preference : UserPreference) : void; + analyzeListing(text : String) : Vehicle; + checkMissingInformation(vehicle : Vehicle) : MissingInformationResult; + retrieveKnowledge(vehicle : Vehicle) : VehicleKnowledge; + estimateCost(vehicle : Vehicle, annualKm : double, fuelPrice : double) : CostEstimate; + checkConstraints(vehicle : Vehicle) : List<ConstraintViolation>; + evaluateVehicle(vehicle : Vehicle) : RiskAssessment; + compareVehicles(first : Vehicle, second : Vehicle) : VehicleComparison; + generateChecklist(vehicle : Vehicle) : InspectionChecklist; + submitClarification(field : String, value : String) : void; + generateReport(vehicle : Vehicle) : EvaluationReport

EvaluationSession — Stores the current user context used as lightweight agent memory. It retains the current vehicle, optional comparison vehicle, user preferences, and clarification history.

Important Attributes: - currentVehicle : Vehicle; - comparisonVehicle : Vehicle; - preference : UserPreference; - clarificationHistory : List<String>

Important Methods: + setCurrentVehicle(vehicle : Vehicle) : void; + setComparisonVehicle(vehicle : Vehicle) : void; + setPreference(preference : UserPreference) : void; + recordClarification(text : String) : void; + getPreference() : UserPreference

UsedCarAgent — Coordinates multi-step agent behavior. It decides which deterministic components and tools are needed, obtains relevant context, invokes the LLM when appropriate, and produces structured results.

Important Attributes: - toolManager : ToolManager; - llmClient : LLMClient; - promptBuilder : PromptBuilder; - responseParser : ResponseParser; - dataQualityChecker : DataQualityChecker; - strategy : EvaluationStrategy

Important Methods: + analyzeListing(text : String) : Vehicle; + assessRisk(vehicle : Vehicle, preference : UserPreference) : RiskAssessment; + compare(first : Vehicle, second : Vehicle, preference : UserPreference) : VehicleComparison; + generateChecklist(vehicle : Vehicle) : InspectionChecklist; + generateClarificationQuestions(result : MissingInformationResult) : List<String>; + generateRecommendation(vehicle : Vehicle, preference : UserPreference) : EvaluationReport; + setStrategy(strategy : EvaluationStrategy) : void

Command Pattern Components

<<interface>> Command — Common command abstraction used so GUI and CLI can invoke the same application operations.

Important Methods: + execute() : Object

AnalyzeListingCommand — Encapsulates a request to analyze pasted listing text.

Important Attributes: - facade : UsedCarApplicationFacade; - listingText : String

Important Methods: + execute() : Object

EvaluateVehicleCommand — Encapsulates a request to perform a vehicle evaluation.

Important Attributes: - facade : UsedCarApplicationFacade; - vehicle : Vehicle

Important Methods: + execute() : Object

CompareVehiclesCommand — Encapsulates a request to compare two vehicles.

Important Attributes: - facade : UsedCarApplicationFacade; - first : Vehicle; - second : Vehicle

Important Methods: + execute() : Object

GenerateChecklistCommand — Encapsulates a request to generate a vehicle-specific inspection checklist.

Important Attributes: - facade : UsedCarApplicationFacade; - vehicle : Vehicle

Important Methods: + execute() : Object

Tool / Deterministic Service Components

ToolManager — Maintains the available tools and provides a single way for the agent to invoke them.

Important Attributes: - tools : Map<String, Tool>

Important Methods: + registerTool(name : String, tool : Tool) : void; + execute(name : String, input : Object) : Object

<<interface>> Tool — Common abstraction for deterministic tools that can be selected and executed by the agent.

Important Methods: + execute(input : Object) : Object

VehicleKnowledgeTool — Retrieves locally stored reliability and inspection notes for a vehicle.

Important Attributes: - repository : VehicleKnowledgeRepository

Important Methods: + execute(input : Object) : Object

CostCalculatorTool — Performs deterministic ownership-cost calculations from user-provided assumptions.

Important Methods: + execute(input : Object) : Object; + calculateAnnualFuelCost(annualKm : double, fuelConsumption : double, fuelPrice : double) : double

ConstraintCheckerTool — Checks a Vehicle against explicit UserPreference constraints and returns violations.

Important Methods: + execute(input : Object) : Object; + check(vehicle : Vehicle, preference : UserPreference) : List<ConstraintViolation>

DataQualityChecker — Performs deterministic missing-field, uncertainty, and direct conflict checks before the agent makes a recommendation.

Important Methods: + check(vehicle : Vehicle) : MissingInformationResult; + checkConflict(manualVehicle : Vehicle, extractedVehicle : Vehicle) : MissingInformationResult

VehicleKnowledgeRepository — Loads and queries the small local JSON/CSV vehicle knowledge base.

Important Attributes: - dataFilePath : String

Important Methods: + load() : void; + findByVehicle(make : String, model : String, year : int) : VehicleKnowledge

LLM Integration Components

<<interface>> LLMClient — Provider-independent interface used by the agent for LLM requests.

Important Methods: + generate(prompt : String) : String

GeminiAdapter — Adapts the provider-specific Gemini API request/response format to the LLMClient interface used by the application.

Important Attributes: - apiKey : String; - endpoint : String

Important Methods: + generate(prompt : String) : String

PromptBuilder — Builds constrained prompts from structured application data so the LLM receives relevant evidence rather than arbitrary application state.

Important Methods: + buildListingExtractionPrompt(text : String) : String; + buildRiskPrompt(vehicle : Vehicle, knowledge : VehicleKnowledge, missing : MissingInformationResult, violations : List<ConstraintViolation>) : String; + buildComparisonPrompt(first : Vehicle, second : Vehicle) : String; + buildChecklistPrompt(vehicle : Vehicle, knowledge : VehicleKnowledge, missing : MissingInformationResult) : String; + buildClarificationPrompt(missing : MissingInformationResult) : String; + buildRecommendationPrompt(vehicle : Vehicle, risk : RiskAssessment, cost : CostEstimate, violations : List<ConstraintViolation>) : String

ResponseParser — Converts LLM responses into structured domain/result objects and detects malformed or incomplete output.

Important Methods: + parseVehicle(response : String) : Vehicle; + parseRiskAssessment(response : String) : RiskAssessment; + parseComparison(response : String) : VehicleComparison; + parseChecklist(response : String) : InspectionChecklist; + parseClarificationQuestions(response : String) : List<String>; + parseReport(response : String) : EvaluationReport

Strategy / Factory Components

<<interface>> EvaluationStrategy — Defines how evidence should be prioritized when producing the final evaluation explanation.

Important Methods: + buildPriorityGuidance(preference : UserPreference) : String

BalancedStrategy — Balances budget, reliability, mileage, risk, and cost information.

Important Methods: + buildPriorityGuidance(preference : UserPreference) : String

BudgetFocusedStrategy — Places greater weight on budget and cost-related evidence while still preserving risk warnings.

Important Methods: + buildPriorityGuidance(preference : UserPreference) : String

ReliabilityFocusedStrategy — Places greater weight on reliability notes, maintenance uncertainty, and inspection risk.

Important Methods: + buildPriorityGuidance(preference : UserPreference) : String

EvaluationStrategyFactory — Creates the appropriate EvaluationStrategy from the user's stated priorities without exposing concrete strategy construction to the rest of the application.

Important Methods: + createStrategy(preference : UserPreference) : EvaluationStrategy

Domain and Result Objects

Vehicle — Represents one used vehicle being evaluated.

Important Attributes: - id : String; - year : int; - make : String; - model : String; - trim : String; - mileage : int; - askingPrice : double; - transmission : String; - fuelType : String; - accidentHistory : String; - maintenanceHistory : String; - fuelConsumption : Double; - notes : String

Important Methods: + updateField(name : String, value : Object) : void

UserPreference — Stores the buyer's explicit constraints and evaluation priorities.

Important Attributes: - maxBudget : double; - maxMileage : Integer; - acceptedTransmissions : Set<String>; - preferredBodyType : String; - reliabilityPriority : int; - fuelEconomyPriority : int; - accidentTolerance : String

CostEstimate — Stores deterministic ownership-cost results.

Important Attributes: - annualFuelUse : double; - annualFuelCost : double; - purchaseTax : Double; - firstYearEstimate : Double

VehicleKnowledge — Stores retrieved local knowledge about reliability concerns and inspection points.

Important Attributes: - summary : String; - commonIssues : List<String>; - inspectionNotes : List<String>; - sourceLabel : String

MissingInformationResult — Stores missing fields, uncertain fields, conflicts, and whether enough information exists to continue.

Important Attributes: - missingFields : List<String>; - uncertainFields : List<String>; - conflicts : List<String>; - sufficient : boolean

ConstraintViolation — Represents one explicit conflict between a Vehicle and the UserPreference profile.

Important Attributes: - constraint : String; - expected : String; - actual : String; - severity : String

RiskAssessment — Stores the structured result of the hybrid risk-evaluation feature.

Important Attributes: - riskLevel : String; - strengths : List<String>; - risks : List<String>; - uncertainties : List<String>; - explanation : String

VehicleComparison — Stores deterministic comparison metrics plus the LLM-generated trade-off explanation.

Important Attributes: - firstVehicleId : String; - secondVehicleId : String; - metricSummary : Map<String, String>; - preferredVehicleId : String; - explanation : String

InspectionChecklist — Stores grouped inspection items and seller questions.

Important Attributes: - sections : Map<String, List<String>>

EvaluationReport — Stores the final personalized advisory recommendation and supporting evidence.

Important Attributes: - category : String; - strengths : List<String>; - risks : List<String>; - missingInformation : List<String>; - costSummary : String; - inspectionPriorities : List<String>; - nextSteps : List<String>; - explanation : String

3.3 Class-Diagram Relationship Checklist for UMLet

The final UMLet class diagram should show the following major relationships. Getter/setter relationships do not need to be drawn; focus on the important structural relationships that explain the design.

- UsedCarGUI and UsedCarCLI depend on UsedCarApplicationFacade.
- AnalyzeListingCommand, EvaluateVehicleCommand, CompareVehiclesCommand, and GenerateChecklistCommand realize the Command interface and depend on UsedCarApplicationFacade.
- UsedCarApplicationFacade has one UsedCarAgent and one EvaluationSession.
- EvaluationSession contains one UserPreference and zero to two Vehicle objects (current vehicle and optional comparison vehicle).
- UsedCarAgent depends on ToolManager, LLMClient, PromptBuilder, ResponseParser, DataQualityChecker, and EvaluationStrategy.
- ToolManager aggregates zero or more Tool implementations.
- VehicleKnowledgeTool, CostCalculatorTool, and ConstraintCheckerTool realize the Tool interface.
- VehicleKnowledgeTool depends on VehicleKnowledgeRepository.
- GeminiAdapter realizes the LLMClient interface and communicates with the external LLM service.
- BalancedStrategy, BudgetFocusedStrategy, and ReliabilityFocusedStrategy realize EvaluationStrategy.
- EvaluationStrategyFactory depends on the concrete strategy classes and returns an EvaluationStrategy.
- EvaluationReport may contain/aggregate a RiskAssessment, CostEstimate, constraint information, and inspection priorities produced during the evaluation.

4. Design Pattern Explanations

The design uses five patterns covered in the course. Each pattern is used to solve an identifiable design problem rather than only to satisfy the numerical requirement.

Pattern 1 — Strategy

Design Problem: Different buyers can prioritize different criteria. A reliability-focused buyer and a budget-focused buyer should not require separate copies of the entire evaluation workflow.

Participating Classes and Roles:

- EvaluationStrategy: Strategy interface that defines buildPriorityGuidance().
- BalancedStrategy: Concrete strategy that balances the available evidence.
- BudgetFocusedStrategy: Concrete strategy that emphasizes price and ownership-cost evidence.
- ReliabilityFocusedStrategy: Concrete strategy that emphasizes reliability, maintenance uncertainty, and inspection risk.
- UsedCarAgent: Context that uses an EvaluationStrategy when preparing a recommendation.

Why the Pattern Is Appropriate: Strategy is appropriate because the varying part of the behavior is the prioritization policy, while the surrounding workflow—retrieval, validation, tool use, and LLM invocation—remains the same.

What Would Be More Difficult Without It: Without Strategy, UsedCarAgent would require large conditional blocks for every priority mode. Adding a new evaluation priority would require modifying the central agent logic, increasing coupling and reducing maintainability.

Pattern 2 — Adapter

Design Problem: The application should not depend directly on a provider-specific LLM API. The external API request and response format may differ from the interface that the rest of the system needs.

Participating Classes and Roles:

- LLMClient: Target interface expected by UsedCarAgent.
- GeminiAdapter: Adapter that converts the application's generate(prompt) call to the provider-specific HTTP/API format and returns a normal String response.
- External LLM Service: Adaptee/external service with provider-specific protocol.
- UsedCarAgent: Client that depends only on LLMClient.

Why the Pattern Is Appropriate: Adapter is appropriate because it isolates provider-specific details from the agent and domain logic. The rest of the system only needs the small LLMClient abstraction.

What Would Be More Difficult Without It: Without Adapter, provider-specific API code would spread into UsedCarAgent and possibly GUI/CLI code. Replacing the LLM provider would then require changes throughout the application.

Pattern 3 — Factory Method / Factory Pattern

Design Problem: The system must select an EvaluationStrategy based on the buyer's priorities without exposing the creation of concrete strategy objects to the rest of the application.

Participating Classes and Roles:

- EvaluationStrategyFactory: Factory component that creates and returns an EvaluationStrategy.
- EvaluationStrategy: Common product interface.
- BalancedStrategy: Concrete product.
- BudgetFocusedStrategy: Concrete product.

• ReliabilityFocusedStrategy: Concrete product.

• UsedCarAgent: Receives the created strategy and uses it.

Why the Pattern Is Appropriate: The factory centralizes creation decisions and keeps the agent focused on evaluation rather than object-construction logic. It also makes strategy creation consistent between GUI and CLI workflows.

What Would Be More Difficult Without It: Without the factory, multiple classes would contain repeated new/if/else logic for selecting concrete strategies. Adding a new strategy would require editing several callers instead of one creation point.

Pattern 4 — Command

Design Problem: Both GUI and CLI must expose major agent functions. The project should not implement a separate copy of each operation for every user interface.

Participating Classes and Roles:

• Command: Command interface with execute().

• AnalyzeListingCommand: Concrete command for F02.

• EvaluateVehicleCommand: Concrete command for F08.

• CompareVehiclesCommand: Concrete command for F09.

• GenerateChecklistCommand: Concrete command for F10.

• UsedCarGUI / UsedCarCLI: Invokers that submit commands.

• UsedCarApplicationFacade: Receiver that performs the requested application operation.

Why the Pattern Is Appropriate: Command is appropriate because a request can be represented as an object and invoked from either interface. This helps satisfy the GUI and CLI requirements without duplicating application logic.

What Would Be More Difficult Without It: Without Command, GUI button handlers and CLI parsing code would directly contain duplicated calls and workflow logic, making the two interfaces more difficult to keep consistent.

Pattern 5 — Facade

Design Problem: The UI should not need to understand UsedCarAgent, ToolManager, repositories, strategies, prompts, parsing, and LLM adapters in order to perform one user action.

Participating Classes and Roles:

• UsedCarApplicationFacade: Facade that exposes high-level operations such as evaluateVehicle(), compareVehicles(), and generateChecklist().

• UsedCarGUI / UsedCarCLI: Clients of the facade.

• UsedCarAgent, EvaluationSession and supporting components: Subsystem hidden behind the facade.

Why the Pattern Is Appropriate: Facade is appropriate because it provides a small, stable application-level API while allowing the internal agent subsystem to remain modular.

What Would Be More Difficult Without It: Without Facade, boundary classes would depend on many internal classes. Small changes to the agent subsystem would therefore cause changes in the GUI and CLI and increase coupling.

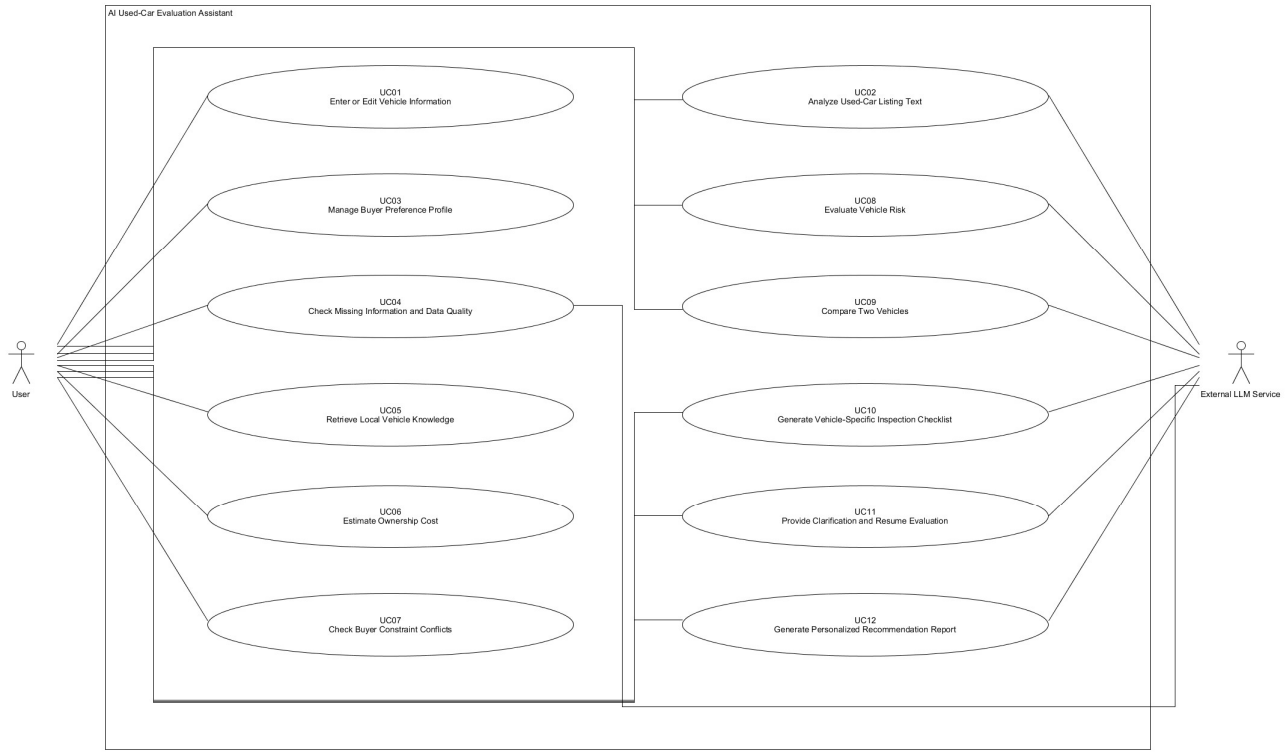
5. Use-Case Diagram

The use-case model describes the system from the external actor's perspective. The primary actor is the User. The External LLM Service is a secondary actor for use cases that require AI-based interpretation or generation. The local JSON/CSV knowledge base is treated as an internal repository rather than an external actor.

5.1 Actors

User: Primary actor. Enters or edits vehicle information, pastes listing text, manages preferences, requests analysis, compares vehicles, answers clarification questions, and reviews recommendations.

External LLM Service: Secondary external actor used for listing interpretation, ambiguity explanation, risk synthesis, comparison explanation, checklist generation, clarification-question generation, and final recommendation text.



6. Detailed Use-Case Descriptions

UC01 — Enter or Edit Vehicle Information

Actor(s): User

Goal: Create or update a validated structured Vehicle record.

Preconditions: The application is running and the vehicle-entry interface is available.

Trigger: The user chooses to add a vehicle or edit the current vehicle.

Main Success Scenario:

1. The user opens the vehicle-entry form or CLI command.
2. The user provides year, make, model, mileage, price, transmission, fuel type, and any optional history information.
3. The system validates required fields and numeric ranges.
4. The system normalizes basic values such as transmission labels.
5. The system creates or updates the Vehicle object.
6. The system stores the Vehicle in EvaluationSession.
7. The system displays the normalized vehicle summary.

Alternative / Exception Flows:

• If a required field is missing, the system identifies the field and asks the user to correct it.

• If a numeric value is invalid or negative, the system rejects the value rather than saving it.

• Optional unknown values remain explicitly unknown rather than being guessed.

Postconditions: A valid Vehicle object is available for later analysis.

Related Feature(s): F01 — Vehicle Information Entry and Management

UC02 — Analyze Used-Car Listing Text

Actor(s): User; External LLM Service

Goal: Extract structured vehicle information from a free-form seller advertisement.

Preconditions: The application is running and the LLM service is configured.

Trigger: The user pastes listing text and selects Analyze Listing.

Main Success Scenario:

1. The user submits the listing text.
2. The GUI/CLI creates AnalyzeListingCommand.
3. The command invokes UsedCarApplicationFacade.analyzeListing().
4. UsedCarAgent uses PromptBuilder to construct a constrained extraction prompt.
5. LLMClient sends the prompt through GeminiAdapter to the external LLM service.
6. ResponseParser converts the response into a draft Vehicle object.
7. DataQualityChecker identifies missing or uncertain extracted fields.
8. The system displays the extracted values for user confirmation.
9. The user confirms or edits the values before they are stored.

Alternative / Exception Flows:

• If the LLM response is malformed, parsing fails safely and the user is offered a retry.

• If the advertisement does not contain a field, the field remains unknown.

• If extracted data conflicts with manually entered data, both values are shown and the user chooses which value to retain.

Postconditions: A user-confirmed Vehicle record is available, together with any identified uncertainty.

Related Feature(s): F02 — Used-Car Listing Text Analysis

UC03 — Manage Buyer Preference Profile

Actor(s): User

Goal: Store explicit buying constraints and evaluation priorities for personalized analysis.

Preconditions: The application is running.

Trigger: The user opens Preferences or uses the corresponding CLI command.

Main Success Scenario:

1. The user enters maximum budget and optional mileage limit.
2. The user specifies acceptable transmission type(s) and optional body-style preference.
3. The user sets reliability and fuel-economy priorities and accident-history tolerance.
4. The system validates the preference values.
5. UsedCarApplicationFacade stores the UserPreference in EvaluationSession.
6. The system displays the saved profile.

Alternative / Exception Flows:

• A negative budget or other invalid value is rejected.

• Unspecified preferences remain neutral and do not become hidden assumptions.

Postconditions: A valid UserPreference profile is stored for later use.

Related Feature(s): F03 — Buyer Preference Profile

UC04 — Check Missing Information and Data Quality

Actor(s): User; External LLM Service when ambiguity exists

Goal: Identify information that is missing, uncertain, or internally inconsistent before evaluation.

Preconditions: A Vehicle record exists.

Trigger: The user requests the check or another evaluation use case invokes it automatically.

Main Success Scenario:

1. The system sends the Vehicle to DataQualityChecker.
2. DataQualityChecker checks required fields, unknown values, and direct conflicts.
3. If a seller statement is ambiguous, UsedCarAgent may request an LLM interpretation of only that statement.
4. The system creates MissingInformationResult.
5. The system displays missing fields, uncertain fields, conflicts, and whether the data is sufficient.

Alternative / Exception Flows:

• If too little information is available, the result is marked insufficient and the system does not produce an overconfident recommendation.

• If two values conflict, both are preserved until the user resolves the conflict.

Postconditions: A MissingInformationResult is available and may trigger UC11.

Related Feature(s): F04 — Missing Information and Data-Quality Detection

UC05 — Retrieve Local Vehicle Knowledge

Actor(s): User

Goal: Retrieve relevant reliability and inspection notes from the local knowledge repository.

Preconditions: A Vehicle with make, model, and year exists and the local data file is available.

Trigger: The user requests knowledge or another evaluation use case invokes retrieval automatically.

Main Success Scenario:

1. UsedCarAgent asks ToolManager to execute VehicleKnowledgeTool.
2. VehicleKnowledgeTool queries VehicleKnowledgeRepository using normalized make, model, and year.
3. The repository returns the best matching VehicleKnowledge entry.
4. The tool returns the structured knowledge to the agent.
5. The system displays or reuses the retrieved notes in later evaluation steps.

Alternative / Exception Flows:

• If no matching record exists, the system returns 'no local data available' rather than fabricating data.

• If the local file cannot be loaded, the system reports the retrieval failure and continues with the remaining available evidence where possible.

Postconditions: A VehicleKnowledge result, or an explicit no-data result, is available.

Related Feature(s): F05 — Local Vehicle Knowledge Retrieval

UC06 — Estimate Ownership Cost

Actor(s): User

Goal: Calculate a transparent ownership-cost estimate from explicit numeric assumptions.

Preconditions: A Vehicle exists and the user can provide the required cost assumptions.

Trigger: The user opens Cost Estimate or invokes estimate-cost.

Main Success Scenario:

1. The user provides annual distance, fuel consumption, fuel price, and any optional tax/allowance values.
2. The system validates all numeric values.
3. UsedCarApplicationFacade invokes CostCalculatorTool.
4. CostCalculatorTool calculates annual fuel use and annual fuel cost using deterministic formulas.
5. Optional tax and first-year estimates are calculated only when their inputs are provided.
6. The system creates and displays CostEstimate.

Alternative / Exception Flows:

• Invalid negative values are rejected.

• If an optional value is not supplied, that part of the estimate is omitted rather than guessed.

Postconditions: A deterministic CostEstimate is available for comparison and recommendation.

Related Feature(s): F06 — Ownership Cost Estimation

UC07 — Check Buyer Constraint Conflicts

Actor(s): User

Goal: Determine whether the vehicle violates explicit buyer constraints.

Preconditions: A Vehicle and UserPreference profile exist.

Trigger: The user requests constraint checking or evaluation invokes it automatically.

Main Success Scenario:

1. UsedCarApplicationFacade invokes ConstraintCheckerTool with Vehicle and UserPreference.
2. The tool compares price with maximum budget.
3. The tool checks mileage when a maximum has been specified.
4. The tool checks transmission and accident-history tolerance when applicable.
5. The tool creates a list of ConstraintViolation objects.
6. The system displays the conflicts and supplies them to later AI evaluation.

Alternative / Exception Flows:

• If a preference has not been specified, that rule is skipped.

• If a vehicle value is unknown, the result is marked unknown and may trigger clarification rather than a false pass.

Postconditions: The application has a structured list of constraint violations.

Related Feature(s): F07 — Buyer Constraint Conflict Detection

UC08 — Evaluate Vehicle Risk

Actor(s): User; External LLM Service

Goal: Produce an explainable risk assessment from structured facts, local knowledge, constraints, and uncertainty.

Preconditions: A Vehicle exists; the system has enough information to begin an evaluation.

Trigger: The user selects Evaluate Vehicle or runs evaluate.

Main Success Scenario:

1. The GUI/CLI creates EvaluateVehicleCommand.
2. The command calls UsedCarApplicationFacade.evaluateVehicle().
3. The facade requests missing-information checking and relevant deterministic tool results.
4. UsedCarAgent retrieves local VehicleKnowledge and constraint results and uses cost data when available.
5. EvaluationStrategyFactory creates the appropriate EvaluationStrategy from the user's priorities.
6. UsedCarAgent applies the strategy guidance to the evidence.
7. PromptBuilder constructs a risk-assessment prompt containing only the structured evidence.
8. LLMClient invokes the external LLM service.
9. ResponseParser validates and converts the response into RiskAssessment.
10. The system displays risk level, strengths, risks, uncertainties, and explanation.

Alternative / Exception Flows:

• If information is insufficient, the system returns Insufficient Information and may invoke UC11 instead of forcing a risk category.

• If the LLM service fails, deterministic results remain visible and the user is informed that AI synthesis is unavailable.

• If the response cannot be parsed, the system can retry with stricter output instructions.

Postconditions: A RiskAssessment is stored or the evaluation is explicitly marked insufficient.

Related Feature(s): F08 — AI-Assisted Vehicle Risk Assessment

UC09 — Compare Two Vehicles

Actor(s): User; External LLM Service

Goal: Compare two vehicles using the same user preferences and explain their trade-offs.

Preconditions: Two different Vehicle records are available.

Trigger: The user selects two vehicles and chooses Compare.

Main Success Scenario:

1. The GUI/CLI creates CompareVehiclesCommand.
2. UsedCarApplicationFacade obtains constraint, cost, and local-knowledge information for both vehicles when available.
3. The system creates deterministic side-by-side metrics.
4. UsedCarAgent builds a structured comparison context.
5. PromptBuilder constructs a comparison prompt that preserves calculated values.
6. LLMClient obtains the trade-off explanation from the external LLM service.
7. ResponseParser creates VehicleComparison.
8. The system displays the side-by-side metrics and explanation.

Alternative / Exception Flows:

• If a metric is missing for one vehicle, the system displays it as unavailable.

• Missing information is not automatically treated as better or worse.

• If the LLM is unavailable, the deterministic comparison table can still be displayed.

Postconditions: A VehicleComparison is available for the user.

Related Feature(s): F09 — Two-Vehicle Comparison

UC10 — Generate Vehicle-Specific Inspection Checklist

Actor(s): User; External LLM Service

Goal: Generate a practical checklist based on the selected vehicle, retrieved knowledge, and missing information.

Preconditions: A Vehicle exists.

Trigger: The user selects Generate Inspection Checklist.

Main Success Scenario:

1. The GUI/CLI creates GenerateChecklistCommand.
2. UsedCarAgent retrieves VehicleKnowledge through ToolManager.

3. DataQualityChecker provides missing/uncertain information.
4. PromptBuilder constructs a checklist prompt using only the available evidence.
5. LLMClient obtains the generated checklist.
6. ResponseParser creates InspectionChecklist.
7. The system displays grouped checklist sections and seller questions.

Alternative / Exception Flows:

• If no local model-specific knowledge exists, the system clearly generates a more general checklist.

• If the LLM call fails, the system can still display deterministic missing-information questions and a basic generic checklist.

Postconditions: An InspectionChecklist is available for review.

Related Feature(s): F10 — Vehicle-Specific Inspection Checklist Generation

UC11 — Provide Clarification and Resume Evaluation

Actor(s): User; External LLM Service

Goal: Ask high-value follow-up questions when important information is missing or ambiguous, then continue the interrupted evaluation.

Preconditions: A MissingInformationResult indicates that clarification would improve the evaluation.

Trigger: The agent determines that one or more important questions should be asked.

Main Success Scenario:

1. UsedCarAgent uses PromptBuilder to prepare a clarification request from MissingInformationResult.
2. LLMClient returns a small prioritized set of clarification questions.
3. ResponseParser validates the questions.
4. The system displays the questions to the user.
5. The user answers, skips, or marks a value unknown.
6. UsedCarApplicationFacade updates the relevant Vehicle or EvaluationSession data.
7. EvaluationSession records the clarification so the same question is not repeatedly asked.
8. The system resumes the original evaluation when sufficient information is available.

Alternative / Exception Flows:

• If the user skips a question, uncertainty is retained.

• If the answer conflicts with existing data, the system displays the conflict and asks which value should be retained.

• If the LLM service is unavailable, deterministic missing-field prompts may be used.

Postconditions: The evaluation context is updated and the interrupted workflow can continue.

Related Feature(s): F11 — Clarification Question and Follow-Up Management

UC12 — Generate Personalized Recommendation Report

Actor(s): User; External LLM Service

Goal: Produce the final advisory recommendation and evidence-based evaluation report.

Preconditions: A Vehicle exists; available deterministic checks and risk assessment have been completed or explicitly marked unavailable.

Trigger: The user selects Generate Recommendation / Report.

Main Success Scenario:

1. UsedCarApplicationFacade verifies that current evidence is available.
2. UsedCarAgent obtains the latest risk, cost, constraint, local-knowledge, and missing-information results.
3. EvaluationStrategyFactory selects the strategy matching the user's priorities.
4. PromptBuilder constructs a recommendation prompt from the structured evidence and strategy guidance.
5. LLMClient requests the final explanation.
6. ResponseParser creates EvaluationReport.
7. The system stores the report in the current session and displays category, strengths, risks, uncertainty, cost summary, inspection priorities, and next steps.

Alternative / Exception Flows:

• If evidence is insufficient, the category is Insufficient Information rather than a confident recommendation.

• If a required tool failed, the report identifies the unavailable evidence.

• If the LLM is unavailable, deterministic results remain accessible and the narrative recommendation is marked unavailable.

Postconditions: A structured EvaluationReport is available for later review.

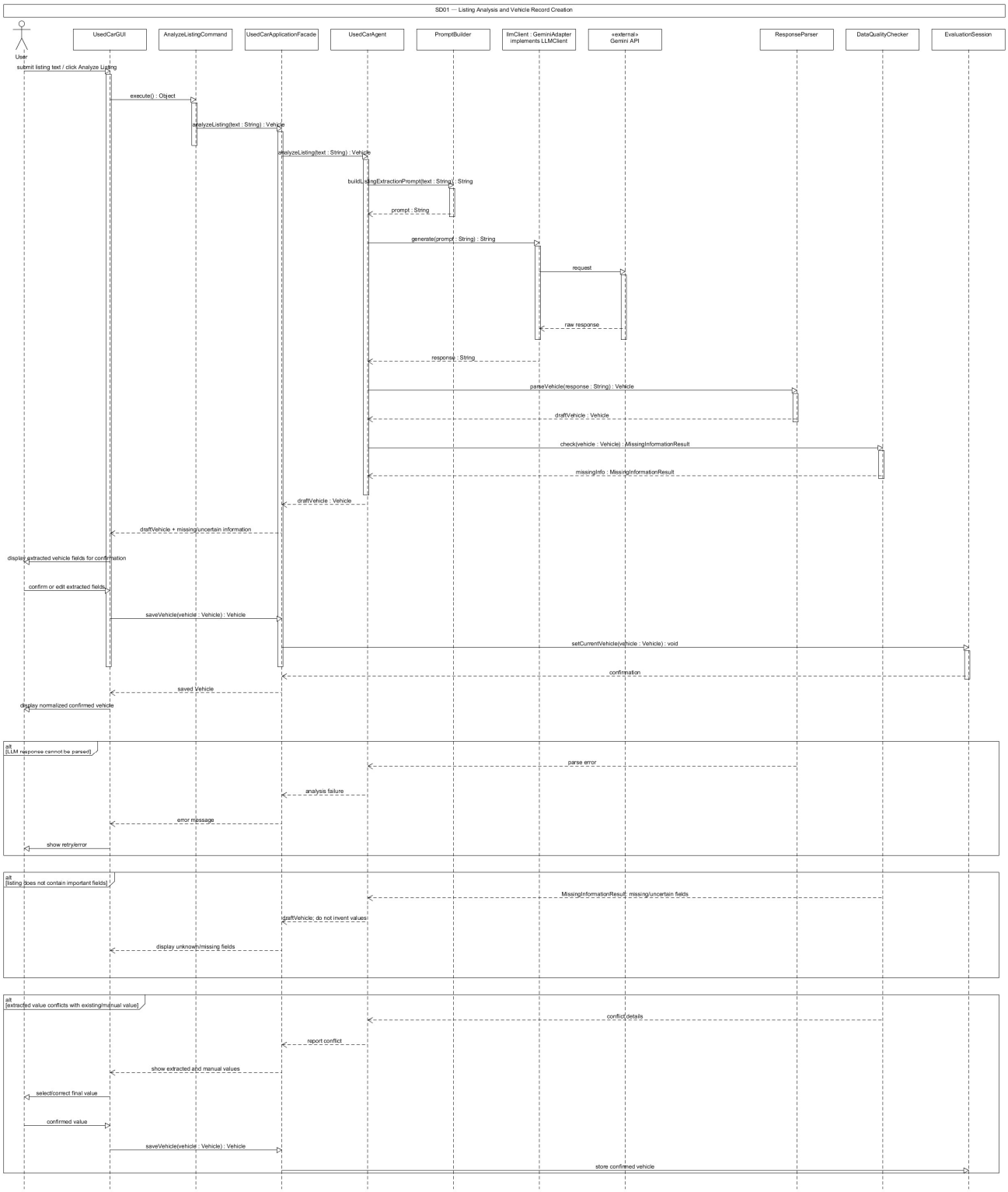
Related Feature(s): F12 — Personalized Purchase Recommendation and Evaluation Report

7. Sequence Diagrams

The sequence diagrams should show runtime collaboration using the actual classes and methods defined in the class diagram. The six diagrams below collectively cover all twelve Task 1 features. Closely related features share a sequence diagram so that the design remains readable and does not create unnecessary implementation complexity.

SD01 — Listing Analysis and Vehicle Record Creation

Coverage: F01, F02 / UC01, UC02


SD01 — Listing Analysis and Vehicle Record Creation
User
UsedCarGUI
AnalyzeListingCommand
UsedCarApplicationFacade
UsedCarAgent
PromptBuilder
llmClient : GeminiAdapter
implements LLMClient
«external»
Gemini API
ResponseParser
DataQualityChecker
EvaluationSession
submit listing text / click Analyze Listing
execute() : Object
analyzeListing(text : String) : Vehicle
analyzeListing(text : String) : Vehicle
buildListingExtractionPrompt(text : String) : String
prompt : String
generate(prompt : String) : String
request
raw response
response : String
parseVehicle(response : String) : Vehicle
draftVehicle : Vehicle
check(vehicle : Vehicle) : MissingInformationResult
missingInfo : MissingInformationResult
draftVehicle : Vehicle
draftVehicle + missing/uncertain information
display extracted vehicle fields for confirmation
confirm or edit extracted fields
saveVehicle(vehicle : Vehicle) : Vehicle
setCurrentVehicle(vehicle : Vehicle) : void
confirmation
saved Vehicle
display normalized confirmed vehicle
alt
[LLM response cannot be parsed]
parse error
analysis failure
error message
show retry/error
alt
[listing does not contain important fields]
MissingInformationResult: missing/uncertain fields
draftVehicle: do not invent values
display unknown/missing fields
alt
[extracted value conflicts with existing/manual value]
conflict details
report conflict
show extracted and manual values
select/correct final value
confirmed value
saveVehicle(vehicle : Vehicle) : Vehicle
store confirmed vehicle

Required Lifelines / Participants: User; UsedCarGUI; AnalyzeListingCommand; UsedCarApplicationFacade; UsedCarAgent; PromptBuilder; LLMClient; GeminiAdapter; ResponseParser; DataQualityChecker; EvaluationSession

Main Interaction to Show:

1. User submits listing text through UsedCarGUI.
2. UsedCarGUI executes AnalyzeListingCommand.execute().
3. AnalyzeListingCommand calls UsedCarApplicationFacade.analyzeListing(text).
4. The facade delegates to UsedCarAgent.analyzeListing(text).
5. UsedCarAgent asks PromptBuilder to buildListingExtractionPrompt(text).
6. UsedCarAgent calls LLMClient.generate(prompt); GeminiAdapter communicates with the external LLM service.
7. ResponseParser.parseVehicle(response) creates a draft Vehicle.
8. DataQualityChecker checks missing/uncertain extracted values.
9. The facade returns the draft to the GUI for confirmation.
10. After confirmation, the facade stores the Vehicle in EvaluationSession.

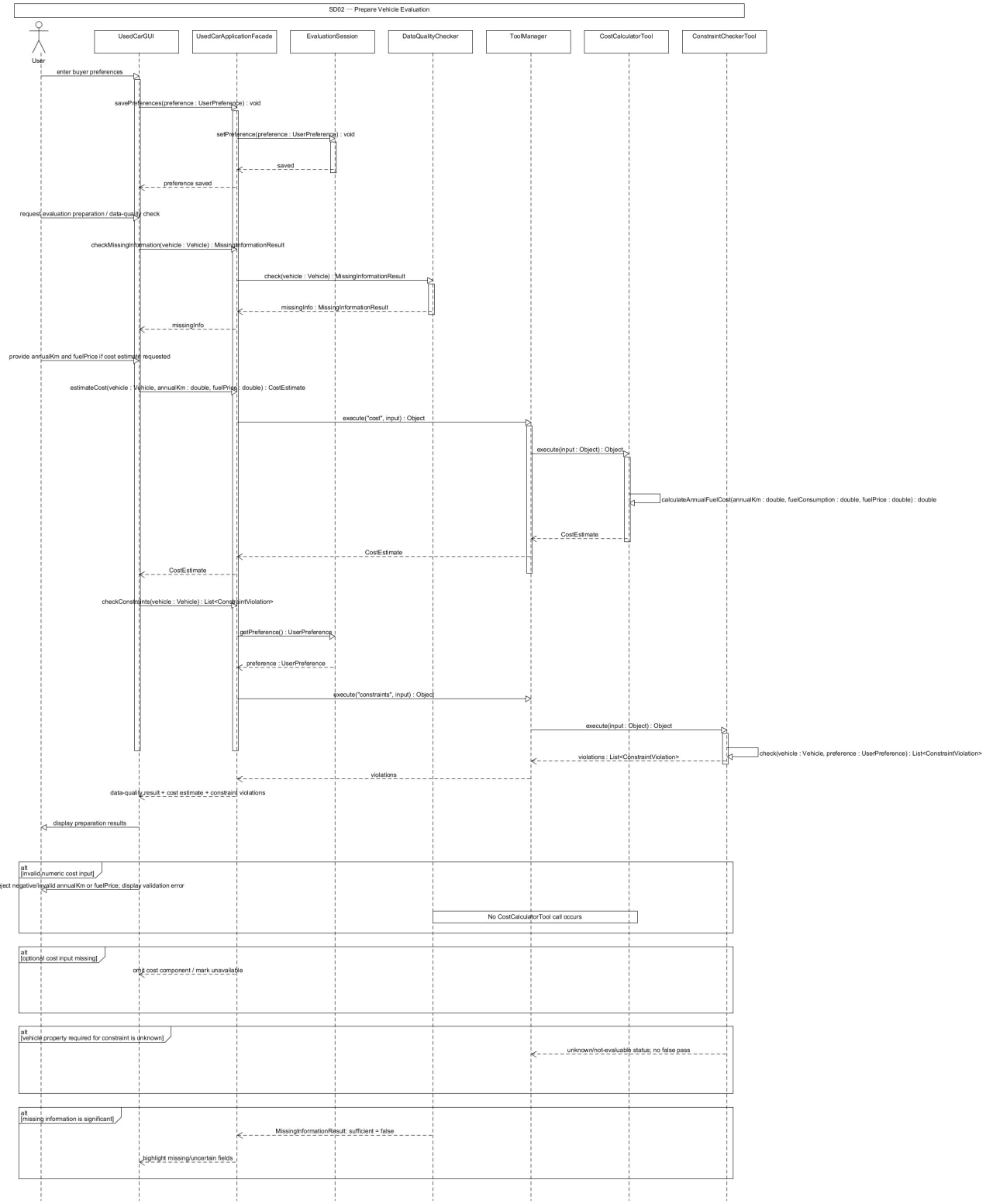
Alternative / Error Fragments to Show:

• alt: malformed LLM response -> ResponseParser reports failure -> GUI displays retry/error.

• alt: extracted/manual data conflict -> DataQualityChecker reports both values -> User resolves conflict.

SD02 — Prepare Vehicle Evaluation (Preferences, Missing Data, Cost, and Constraints)

Coverage: F03, F04, F06, F07 / UC03, UC04, UC06, UC07



Required Lifelines / Participants: User; UsedCarGUI; UsedCarApplicationFacade; EvaluationSession; DataQualityChecker; ToolManager; CostCalculatorTool; ConstraintCheckerTool

Main Interaction to Show:

1. User saves UserPreference through the GUI.
2. UsedCarApplicationFacade stores it in EvaluationSession.
3. User requests missing-information check and/or evaluation preparation.
4. The facade calls DataQualityChecker.check(vehicle).
5. When cost inputs are provided, the facade asks ToolManager to execute CostCalculatorTool.
6. ToolManager returns CostEstimate.
7. The facade asks ToolManager to execute ConstraintCheckerTool with Vehicle and UserPreference.
8. ToolManager returns List<ConstraintViolation>.
9. The facade returns the combined deterministic results to the GUI and makes them available to the agent.

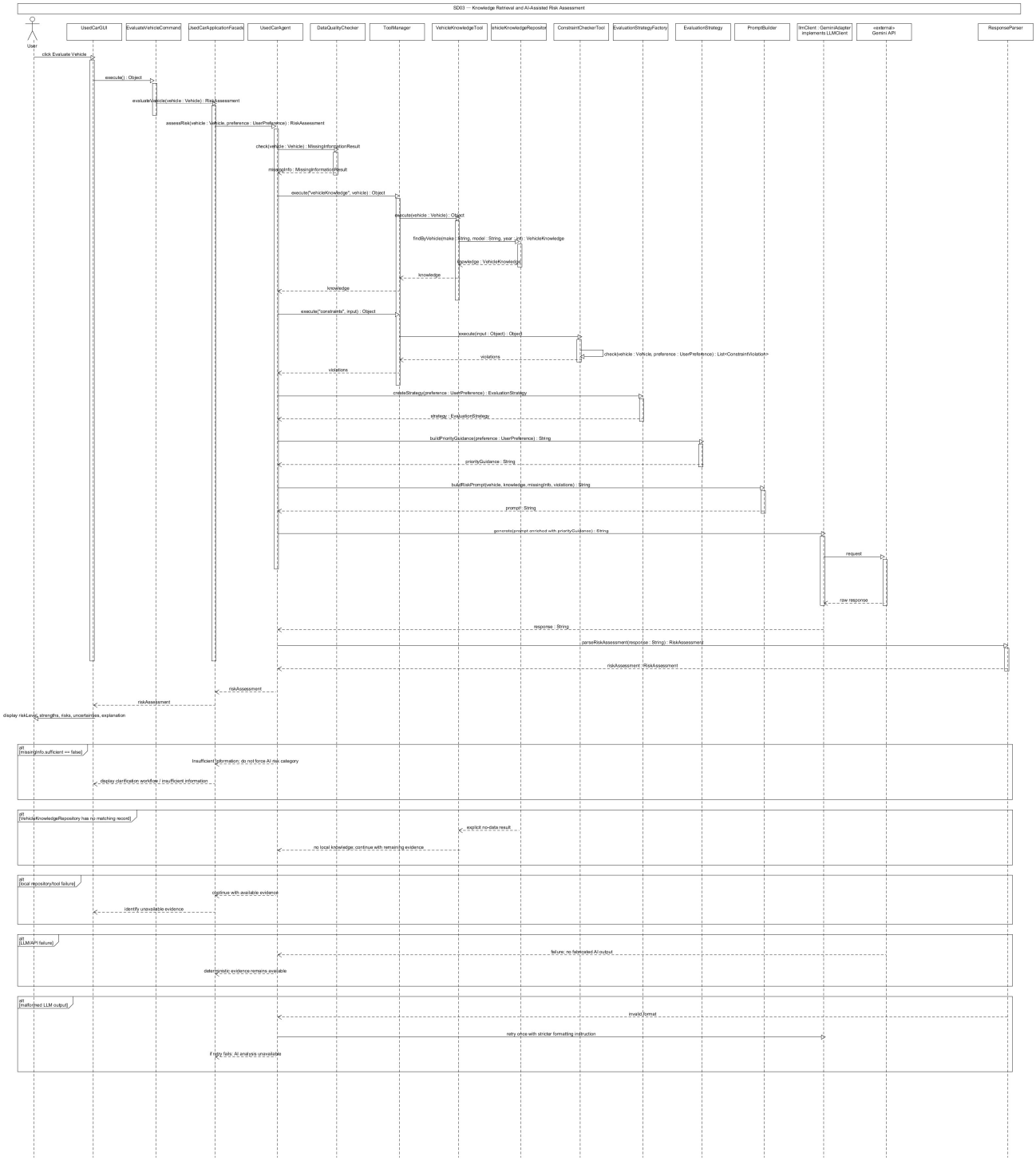
Alternative / Error Fragments to Show:

• alt: invalid numeric cost input -> reject before tool execution.

• alt: unknown vehicle value -> constraint is marked unknown rather than passed.

SD03 — Knowledge Retrieval and AI-Assisted Risk Assessment

Coverage: F05, F08 / UC05, UC08



Required Lifelines / Participants: User; UsedCarGUI; EvaluateVehicleCommand; UsedCarApplicationFacade; UsedCarAgent; DataQualityChecker; ToolManager; VehicleKnowledgeTool; VehicleKnowledgeRepository; ConstraintCheckerTool; EvaluationStrategyFactory; EvaluationStrategy; PromptBuilder; LLMClient; GeminiAdapter; ResponseParser

Main Interaction to Show:

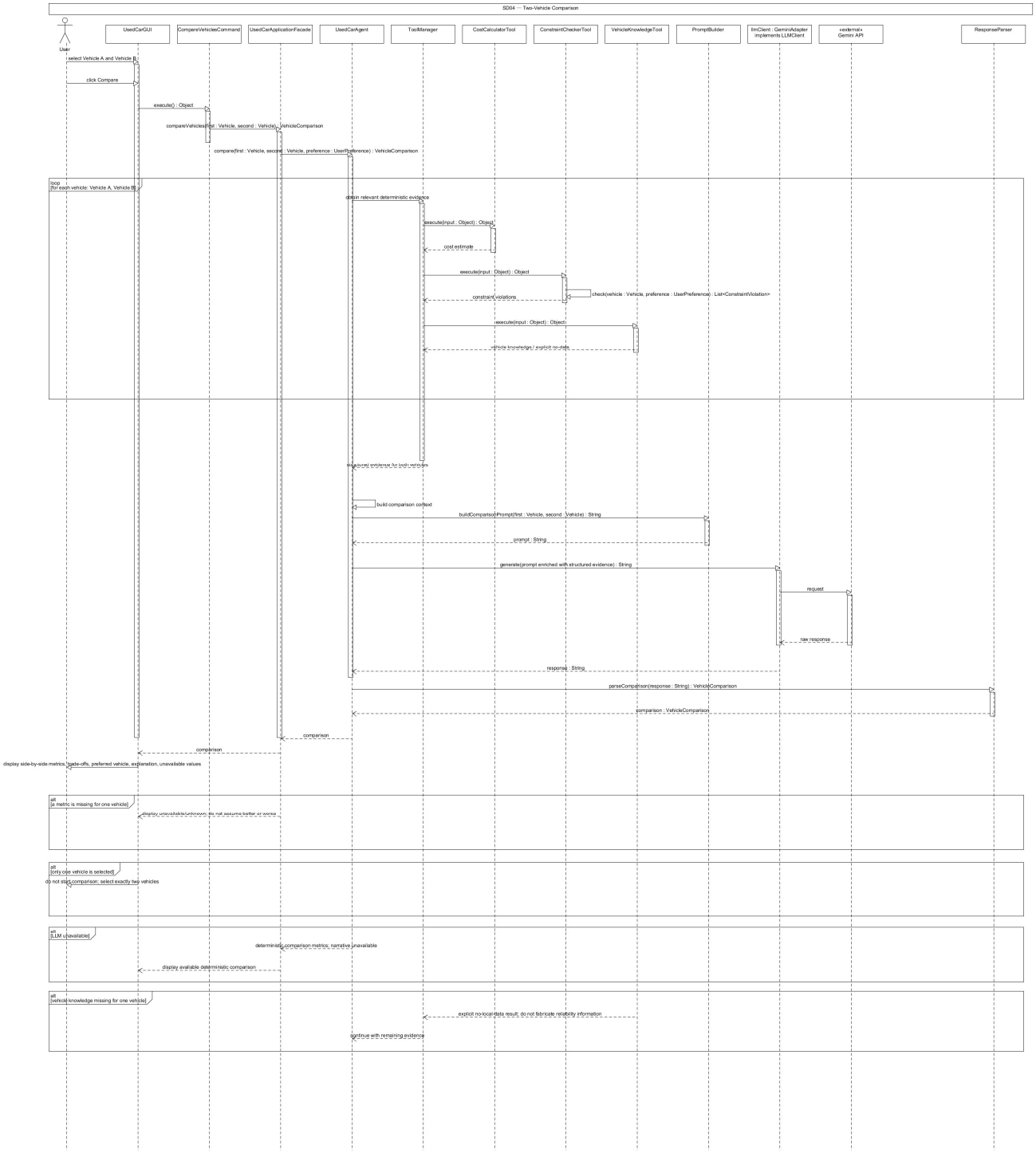
1. User selects Evaluate Vehicle.
2. UsedCarGUI executes EvaluateVehicleCommand.
3. The command invokes UsedCarApplicationFacade.evaluateVehicle(vehicle).
4. UsedCarAgent obtains missing-information status from DataQualityChecker.
5. UsedCarAgent asks ToolManager to execute VehicleKnowledgeTool; the tool queries VehicleKnowledgeRepository.
6. UsedCarAgent asks ToolManager for current constraint results.
7. EvaluationStrategyFactory.createStrategy(preference) returns the appropriate strategy.
8. UsedCarAgent applies EvaluationStrategy.buildPriorityGuidance(preference).
9. PromptBuilder.buildRiskPrompt(...) creates the evidence-grounded prompt.
10. LLMClient.generate(prompt) invokes the external LLM through GeminiAdapter.
11. ResponseParser.parseRiskAssessment(response) returns RiskAssessment.
12. The facade returns the RiskAssessment to the GUI.

Alternative / Error Fragments to Show:

- alt: insufficient information -> UsedCarAgent stops risk synthesis and triggers clarification/Insufficient Information.
- alt: no local knowledge -> continue with explicit no-data status.
- alt: LLM failure -> deterministic evidence is returned with AI analysis unavailable.

SD04 — Two-Vehicle Comparison

Coverage: F09 / UC09



Required Lifelines / Participants: User; UsedCarGUI; CompareVehiclesCommand; UsedCarApplicationFacade; UsedCarAgent; ToolManager; CostCalculatorTool; ConstraintCheckerTool; VehicleKnowledgeTool; PromptBuilder; LLMClient; GeminiAdapter; ResponseParser

Main Interaction to Show:

1. User selects two Vehicle records and chooses Compare.
2. UsedCarGUI executes CompareVehiclesCommand.
3. The facade obtains deterministic metrics and tool results for both vehicles.
4. UsedCarAgent creates a structured side-by-side comparison context.
5. PromptBuilder.buildComparisonPrompt(first, second) constructs the prompt using the fixed metrics.
6. LLMClient invokes the external LLM service.
7. ResponseParser.parseComparison(response) returns VehicleComparison.
8. The facade returns the result to the GUI for display.

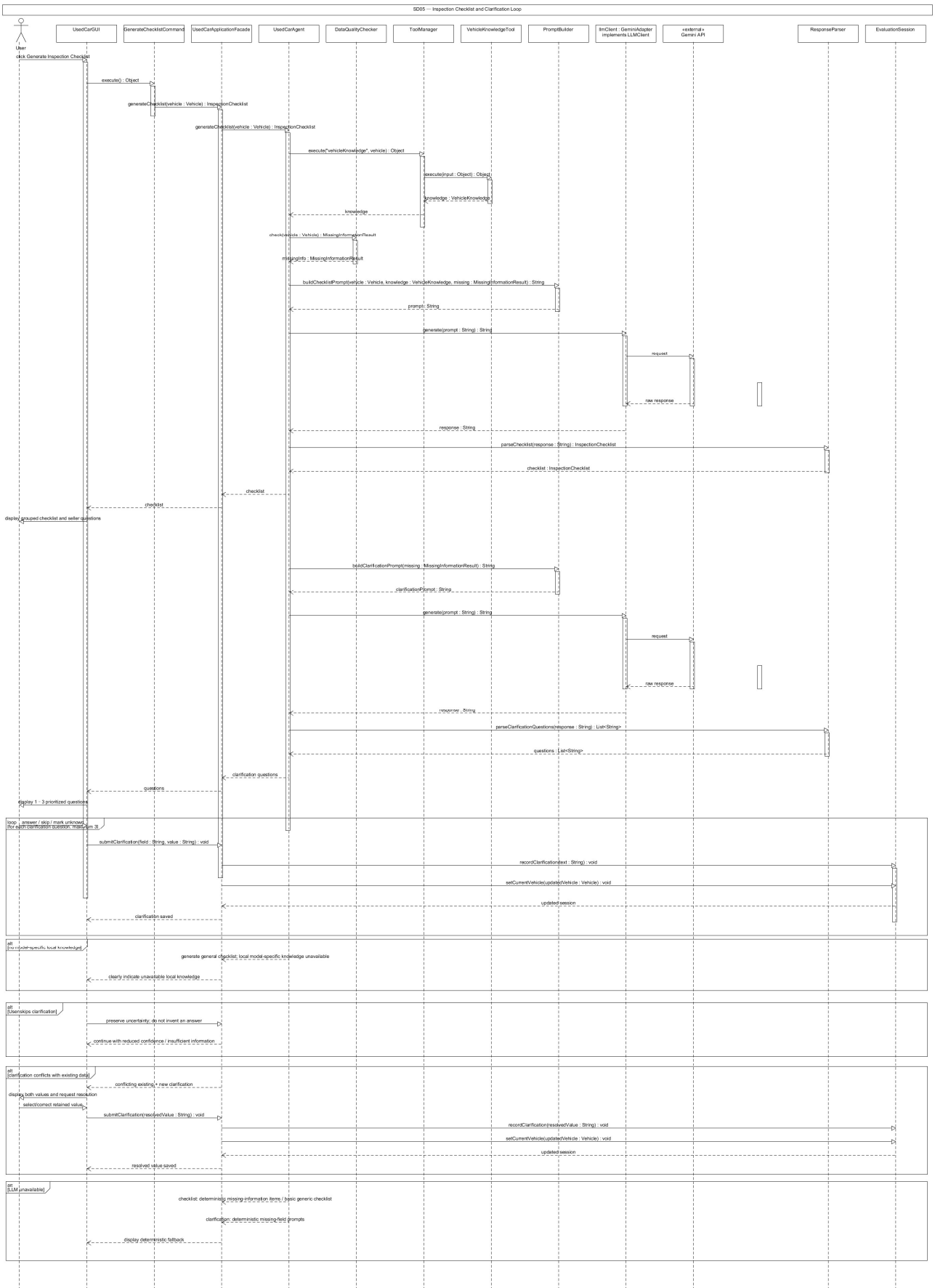
Alternative / Error Fragments to Show:

• alt: missing metric -> display unavailable and preserve uncertainty.

• alt: LLM failure -> show deterministic comparison without narrative trade-off explanation.

SD05 — Inspection Checklist and Clarification Loop

Coverage: F10, F11 / UC10, UC11


SD05 — Inspection Checklist and Clarification Loop
User
UsedCarGUI
GenerateChecklistCommand
UsedCarApplicationFacade
UsedCarAgent
DataQualityChecker
ToolManager
VehicleKnowledgeTool
PromptBuilder
llmClient : GeminiAdapter
implements LLMClient
«external»
Gemini API
ResponseParser
EvaluationSession
click Generate Inspection Checklist
execute() : Object
generateChecklist(vehicle : Vehicle) : InspectionChecklist
generateChecklist(vehicle : Vehicle) : InspectionChecklist
execute("vehicleKnowledge", vehicle) : Object
execute(input : Object) : Object
knowledge : VehicleKnowledge
knowledge
check(vehicle : Vehicle) : MissingInformationResult
missingInfo : MissingInformationResult
buildChecklistPrompt(vehicle : Vehicle, knowledge : VehicleKnowledge, missing : MissingInformationResult) : String
prompt : String
generate(prompt : String) : String
request
raw response
response : String
parseChecklist(response : String) : InspectionChecklist
checklist : InspectionChecklist
checklist
checklist
display grouped checklist and seller questions
buildClarificationPrompt(missing : MissingInformationResult) : String
clarificationPrompt : String
generate(prompt : String) : String
request
raw response
response : String
parseClarificationQuestions(response : String) : List<String>
questions : List<String>
clarification questions
questions
display 1 – 3 prioritized questions
loop [answer / skip / mark unknown]
[for each clarification question, max/turn 3]
submitClarification(field : String, value : String) : void
recordClarification(text : String) : void
setCurrentVehicle(updatedVehicle : Vehicle) : void
updated session
clarification saved
alt
[dealer-specific local knowledge]
generate general checklist; local model-specific knowledge unavailable
clearly indicate unavailable local knowledge
alt
[User skips clarification]
preserve uncertainty; do not invent an answer
continue with reduced confidence / insufficient information
alt
[clarification conflicts with existing data]
conflicting existing + new clarification
display both values and request resolution
select/correct retained value
submitClarification(resolvedValue : String) : void
recordClarification(resolvedValue : String) : void
setCurrentVehicle(updatedVehicle : Vehicle) : void
updated session
resolved value saved
alt
[LLM unavailable]
checklist: deterministic missing-information items / basic generic checklist
clarification: deterministic missing-field prompts
display deterministic fallback

Required Lifelines / Participants: User; UsedCarGUI; GenerateChecklistCommand; UsedCarApplicationFacade; UsedCarAgent; DataQualityChecker; ToolManager; VehicleKnowledgeTool; PromptBuilder; LLMClient; GeminiAdapter; ResponseParser; EvaluationSession

Main Interaction to Show:

1. User requests an inspection checklist.
2. UsedCarGUI executes GenerateChecklistCommand.
3. UsedCarAgent retrieves local knowledge and current MissingInformationResult.
4. PromptBuilder.buildChecklistPrompt(...) is sent through LLMClient.
5. ResponseParser creates InspectionChecklist and the GUI displays it.
6. If important information is still missing, UsedCarAgent builds a clarification prompt.
7. LLMClient returns prioritized questions and ResponseParser validates them.
8. GUI displays the questions; User responds.
9. UsedCarApplicationFacade updates the Vehicle/Session and EvaluationSession records the clarification.
10. The system resumes the interrupted evaluation when appropriate.

Alternative / Error Fragments to Show:

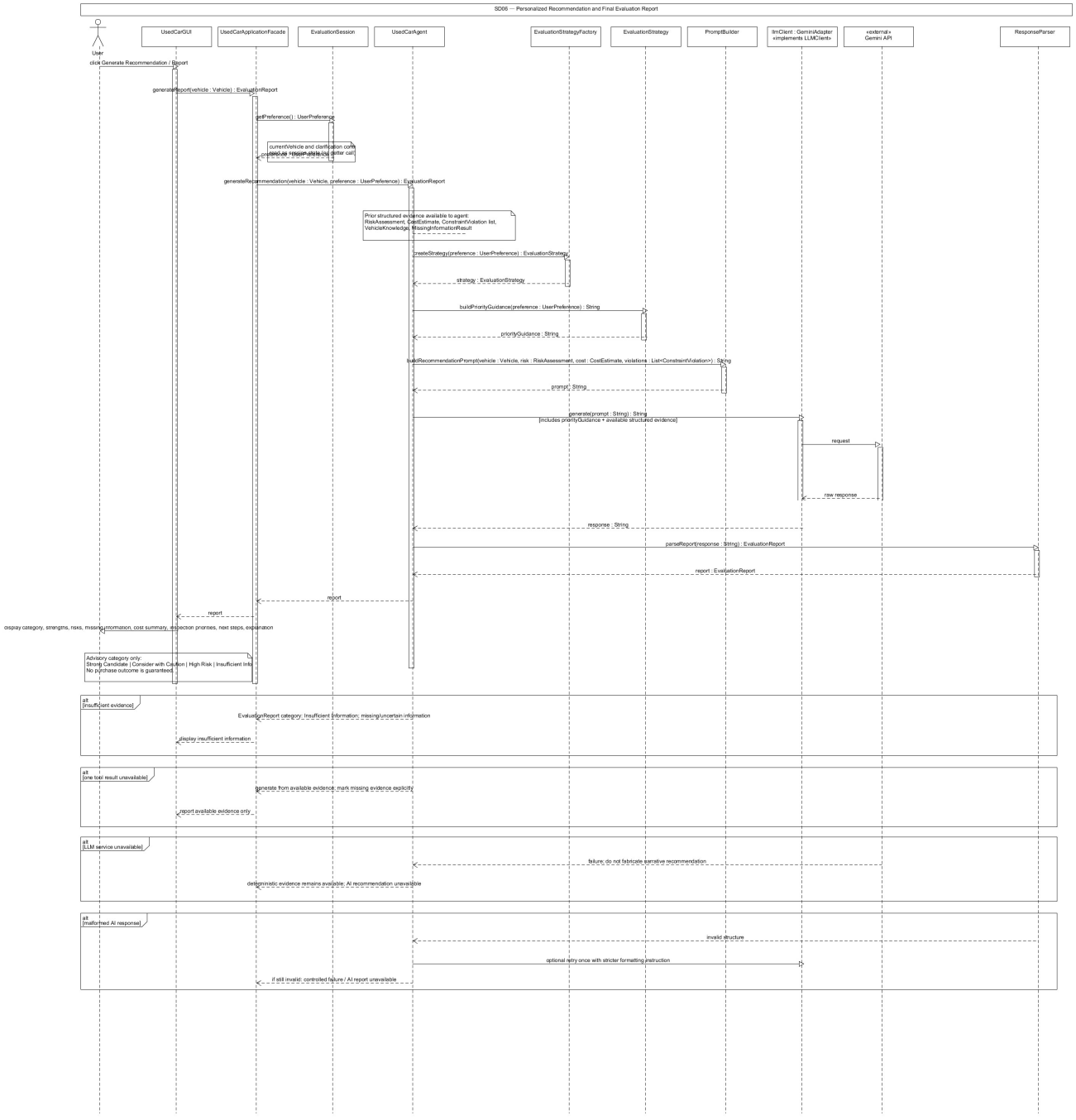
• alt: user skips a question -> preserve unknown status.

• alt: answer conflicts with existing data -> display conflict for user resolution.

• alt: LLM unavailable -> show deterministic missing-information questions/basic checklist.

SD06 — Personalized Recommendation and Final Evaluation Report

Coverage: F12 / UC12



Required Lifelines / Participants: User; UsedCarGUI; UsedCarApplicationFacade; UsedCarAgent; EvaluationSession; EvaluationStrategyFactory; EvaluationStrategy; ToolManager; PromptBuilder; LLMClient; GeminiAdapter; ResponseParser

Main Interaction to Show:

1. User selects Generate Recommendation / Report.
2. UsedCarApplicationFacade asks EvaluationSession for the latest Vehicle and UserPreference.
3. UsedCarAgent obtains the most recent deterministic evidence and RiskAssessment.
4. EvaluationStrategyFactory returns the strategy matching the user's priorities.
5. EvaluationStrategy provides priority guidance.
6. PromptBuilder.buildRecommendationPrompt(...) creates a final evidence-grounded prompt.
7. LLMClient generates the recommendation through GeminiAdapter.
8. ResponseParser.parseReport(response) creates EvaluationReport.
9. The facade stores/returns the report.
10. UsedCarGUI displays the final advisory category, evidence, uncertainty, and next steps.

Alternative / Error Fragments to Show:

• alt: insufficient evidence -> report category is Insufficient Information.

• alt: tool result unavailable -> report identifies the missing evidence.

• alt: LLM unavailable -> deterministic evidence remains viewable; narrative report is unavailable.

7.1 Sequence-Diagram Consistency Rules

- Every object shown in a sequence diagram must correspond to a class/interface, external actor/service, or boundary defined in this Task 2 design.
- Method-call names should match the important methods listed in Section 3 whenever a named method is shown.
- The initiating User actor should appear on the left, followed by the GUI/CLI boundary, application/facade or command object, agent/services/tools, and external LLM service where applicable.
- Return messages should show important structured results such as Vehicle, MissingInformationResult, VehicleKnowledge, CostEstimate, RiskAssessment, VehicleComparison, InspectionChecklist, and EvaluationReport.
- Use UML alternative fragments for important failure paths such as invalid input, unavailable local knowledge, malformed LLM output, LLM service failure, and insufficient information.

7.2 Scope and Stage 2/3 Feasibility Note

This design intentionally satisfies the required software-design and agent-design concepts without introducing unnecessary external dependencies. The implementation can remain a Java application with a simple GUI, CLI, a small local JSON/CSV knowledge repository, deterministic tools, and one LLM API adapter. Most additional classes in the design are small interfaces, commands, strategies, or data/result objects. This keeps Stage 2 implementation manageable while creating clear deterministic components for conventional automated testing and clearly separated LLM/agent behavior for Stage 3 behavioral testing.

8. Feature-to-Design Traceability Table

Feature	Description	Type	Related Use Case	Classes	Key Methods	Sequence Diagram	Design Pattern(s)
F01	Vehicle Information Entry and Management	Deterministic	UC01 Enter or Edit Vehicle Information	UsedCarGUI, UsedCarApplicationFacade, EvaluationSession, Vehicle	saveVehicle(), setCurrentVehicle(), updateField()	SD01	Facade
F02	Used-Car Listing Text Analysis	AI + deterministic validation	UC02 Analyze Used-Car Listing Text	AnalyzeListingCommand, UsedCarApplicationFacade, UsedCarAgent, PromptBuilder, LLMClient/GeminiAdapter, ResponseParser, DataQualityChecker, Vehicle	execute(), analyzeListing(), buildListingExtractionPrompt(), generate(), parseVehicle(), check()	SD01	Command, Adapter, Facade
F03	Buyer Preference Profile	Deterministic	UC03 Manage Buyer Preference Profile	UsedCarGUI, UsedCarApplicationFacade, EvaluationSession, UserPreference	savePreferences(), setPreference(), getPreference()	SD02	Facade
F04	Missing Information and Data-Quality Detection	Hybrid	UC04 Check Missing Information and Data Quality	UsedCarApplicationFacade, DataQualityChecker, MissingInformationResult, UsedCarAgent, LLMClient/GeminiAdapter	checkMissingInformation(), check(), checkConflict(), generate() when ambiguity requires AI	SD02	Facade, Adapter
F05	Local Vehicle Knowledge Retrieval	Deterministic retrieval	UC05 Retrieve Local Vehicle Knowledge	UsedCarApplicationFacade, UsedCarAgent, ToolManager, VehicleKnowledgeTool, VehicleKnowledgeRepository, VehicleKnowledge	retrieveKnowledge(), execute(), findByVehicle()	SD03	Facade
F06	Ownership Cost Estimation	Deterministic	UC06 Estimate Ownership Cost	UsedCarApplicationFacade, ToolManager, CostCalculatorTool, CostEstimate	estimateCost(), execute(), calculateAnnualFuelCost()	SD02	Facade
F07	Buyer Constraint Conflict Detection	Deterministic	UC07 Check Buyer Constraint Conflicts	UsedCarApplicationFacade, EvaluationSession, ToolManager, ConstraintCheckerTool, UserPreference, ConstraintViolation	checkConstraints(), getPreference(), execute(), check()	SD02	Facade

Feature	Description	Type	Related Use Case	Classes	Key Methods	Sequence Diagram	Design Pattern(s)
F08	AI-Assisted Vehicle Risk Assessment	Hybrid	UC08 Evaluate Vehicle Risk	EvaluateVehicleCommand, UsedCarApplicationFacade, UsedCarAgent, DataQualityChecker, ToolManager, EvaluationStrategyFactory, EvaluationStrategy, PromptBuilder, LLMClient/GeminiAdapter, ResponseParser, RiskAssessment	execute(), evaluateVehicle(), assessRisk(), check(), createStrategy(), buildPriorityGuidance(), buildRiskPrompt(), generate(), parseRiskAssessment()	SD03	Strategy, Factory, Command, Adapter, Facade
F09	Two-Vehicle Comparison	Hybrid	UC09 Compare Two Vehicles	CompareVehiclesCommand, UsedCarApplicationFacade, UsedCarAgent, ToolManager, CostCalculatorTool, ConstraintCheckerTool, VehicleKnowledgeTool, PromptBuilder, LLMClient/GeminiAdapter, ResponseParser, VehicleComparison	execute(), compareVehicles(), compare(), ToolManager.execute(), buildComparisonPrompt(), generate(), parseComparison()	SD04	Command, Adapter, Facade
F10	Vehicle-Specific Inspection Checklist Generation	Hybrid	UC10 Generate Vehicle-Specific Inspection Checklist	GenerateChecklistCommand, UsedCarApplicationFacade, UsedCarAgent, DataQualityChecker, ToolManager, VehicleKnowledgeTool, PromptBuilder, LLMClient/GeminiAdapter, ResponseParser, InspectionChecklist	execute(), generateChecklist(), check(), ToolManager.execute(), buildChecklistPrompt(), generate(), parseChecklist()	SD05	Command, Adapter, Facade
F11	Clarification Question and Follow-Up Management	AI + deterministic storage/validation	UC11 Provide Clarification and Resume Evaluation	UsedCarAgent, PromptBuilder, LLMClient/GeminiAdapter, ResponseParser, UsedCarApplicationFacade, EvaluationSession, MissingInformationResult, Vehicle	generateClarificationQuestions(), buildClarificationPrompt(), generate(), parseClarificationQuestions(), submitClarification(), recordClarification(), setCurrentVehicle()	SD05	Adapter, Facade
F12	Personalized Purchase Recommendation and Evaluation Report	Hybrid	UC12 Generate Personalized Recommendation Report	UsedCarApplicationFacade, EvaluationSession, UsedCarAgent, EvaluationStrategyFactory, EvaluationStrategy, PromptBuilder, LLMClient/GeminiAdapter, ResponseParser, EvaluationReport	generateReport(), getPreference(), generateRecommendation(), createStrategy(), buildPriorityGuidance(), buildRecommendationPrompt(), generate(), parseReport()	SD06	Strategy, Factory, Adapter, Facade

9. Feature Implementation Explanations

In addition to the Feature-to-Design Traceability Table in Task 3, this section explains how each proposed feature is realized by the software design. For every feature, the related use case and sequence diagram are identified, followed by the participating classes, important methods, and the runtime collaboration that implements the behavior. The descriptions use the same class/interface names and method names defined in the Task 2 class design and preserve the separation between deterministic Java logic, local tool/repository operations, session state, and Gemini-assisted reasoning. Where multiple features share one sequence diagram, the referenced diagram represents the common runtime structure rather than requiring a duplicate diagram for every feature.

F01 — Vehicle Information Entry and Management

Related Use Case: UC01 — Enter or Edit Vehicle Information

Related Sequence Diagram: SD01 — Listing Analysis and Vehicle Record Creation

Classes involved:

- **UsedCarGUI / UsedCarCLI** — boundary components that collect vehicle fields and present the normalized vehicle record; both reuse the same application layer rather than duplicating business logic.
- **UsedCarApplicationFacade** — provides the application-level saveVehicle(vehicle) operation and hides session-management details from the user interfaces.
- **Vehicle** — encapsulates the structured vehicle data, including year, make, model, mileage, asking price, transmission, fuel type, history information, fuel consumption, and notes.
- **EvaluationSession** — stores the validated Vehicle as the current vehicle so later evaluation features reuse the same structured state.

Important methods:

- UsedCarApplicationFacade.saveVehicle(vehicle : Vehicle) : Vehicle
- EvaluationSession.setCurrentVehicle(vehicle : Vehicle) : void
- Vehicle.updateField(name : String, value : Object) : void

Execution:

The user enters or edits the vehicle through the GUI or CLI. The boundary layer constructs or updates a Vehicle object and sends it to UsedCarApplicationFacade.saveVehicle(vehicle), which is the stable application entry point for this operation. Required fields and numeric values are validated deterministically before the vehicle becomes part of the evaluation state. Vehicle.updateField(name, value) supports controlled edits without exposing the internal representation of the object. After validation, EvaluationSession.setCurrentVehicle(vehicle) stores the record as the current context. The GUI/CLI then displays the normalized values to the user. SD01 shows the vehicle-record creation and confirmation portion of this workflow, which is shared with listing analysis.

Alternative / error behavior: Missing required fields or invalid/negative numeric values are rejected instead of being stored. Optional unknown values remain explicitly unknown. Gemini is not used for manual vehicle entry, so this feature stays deterministic and directly testable.

F02 — Used-Car Listing Text Analysis

Related Use Case: UC02 — Analyze Used-Car Listing Text

Related Sequence Diagram: SD01 — Listing Analysis and Vehicle Record Creation

Classes involved:

- **UsedCarGUI / UsedCarCLI** — accepts the seller advertisement text and displays the extracted draft for user confirmation.

- **AnalyzeListingCommand** — encapsulates the listing-analysis request so the same operation can be invoked from GUI or CLI.
- **UsedCarApplicationFacade** — receives the high-level request and delegates the AI-assisted extraction workflow to the agent.
- **UsedCarAgent** — coordinates prompt creation, LLM invocation, parsing, and data-quality checking.
- **PromptBuilder** — constructs a constrained extraction prompt from the raw listing text.
- **LLMClient / GeminiAdapter** — isolates provider-specific Gemini communication behind the provider-independent generate(prompt) interface.
- **ResponseParser** — converts the LLM response into a structured Vehicle object and rejects malformed output.
- **DataQualityChecker** — identifies missing, uncertain, or conflicting extracted values.
- **EvaluationSession** — stores the Vehicle only after the user confirms or corrects the extracted values.

Important methods:

- AnalyzeListingCommand.execute() : Object
- UsedCarApplicationFacade.analyzeListing(text : String) : Vehicle
- UsedCarAgent.analyzeListing(text : String) : Vehicle
- PromptBuilder.buildListingExtractionPrompt(text : String) : String
- LLMClient.generate(prompt : String) : String
- GeminiAdapter.generate(prompt : String) : String
- ResponseParser.parseVehicle(response : String) : Vehicle
- DataQualityChecker.check(vehicle : Vehicle) : MissingInformationResult
- EvaluationSession.setCurrentVehicle(vehicle : Vehicle) : void

Execution:

When the user chooses Analyze Listing, the boundary creates AnalyzeListingCommand and execute() invokes UsedCarApplicationFacade.analyzeListing(text). The facade delegates to UsedCarAgent.analyzeListing(text). The agent asks PromptBuilder.buildListingExtractionPrompt(text) to create a structured extraction request and calls LLMClient.generate(prompt); at runtime, GeminiAdapter implements LLMClient and performs the provider-specific Gemini request. ResponseParser.parseVehicle(response) converts the returned text into a draft Vehicle rather than allowing free-form model output to flow through the rest of the program. DataQualityChecker.check(vehicle) then marks missing or uncertain fields. The draft is returned to the GUI for confirmation, and only user-confirmed values are stored in EvaluationSession.

Alternative / error behavior: If Gemini returns malformed output, ResponseParser fails safely and the user receives a retry/error message. Fields not present in the advertisement remain unknown rather than being invented. If extracted values conflict with manually entered values, both are displayed and the user decides which value to retain.

F03 — Buyer Preference Profile

Related Use Case: UC03 — Manage Buyer Preference Profile

Related Sequence Diagram: SD02 — Prepare Vehicle Evaluation

Classes involved:

- **UsedCarGUI / UsedCarCLI** — collects the buyer's explicit budget, mileage, transmission, reliability, fuel-economy, body-type, and accident-tolerance preferences.
- **UsedCarApplicationFacade** — provides the single savePreferences(preference) application operation.

- **UserPreference** — encapsulates the buyer's explicit constraints and prioritization values.
- **EvaluationSession** — stores and returns the UserPreference as lightweight session memory for later evaluation features.

Important methods:

- UsedCarApplicationFacade.savePreferences(preference : UserPreference) : void
- EvaluationSession.setPreference(preference : UserPreference) : void
- EvaluationSession.getPreference() : UserPreference

Execution:

The user enters buyer constraints and priorities through the GUI or CLI. UsedCarApplicationFacade.savePreferences(preference) validates the request at the application boundary and stores the resulting UserPreference in EvaluationSession through setPreference(preference). Later features retrieve the exact same profile with getPreference(), so cost/risk interpretation, constraint checking, comparisons, strategy selection, and final recommendations all use one consistent source of user intent. SD02 shows the preference being saved before the deterministic preparation steps. Because GUI and CLI depend on the facade instead of the session object directly, the preference workflow remains independent of the presentation technology.

Alternative / error behavior: Invalid values such as a negative budget are rejected. A preference the user does not specify remains neutral; the software does not silently create a hidden preference. No LLM is required for storing the profile.

F04 — Missing Information and Data-Quality Detection

Related Use Case: UC04 — Check Missing Information and Data Quality

Related Sequence Diagram: SD02 — Prepare Vehicle Evaluation

Classes involved:

- **UsedCarApplicationFacade** — exposes checkMissingInformation(vehicle) as the application-level operation.
- **DataQualityChecker** — performs the deterministic required-field, uncertainty, and direct-conflict checks.
- **MissingInformationResult** — stores missing fields, uncertain fields, conflicts, and the sufficient flag.
- **UsedCarAgent** — may coordinate a narrowly scoped AI interpretation when seller wording is genuinely ambiguous.
- **LLMClient / GeminiAdapter** — provides optional Gemini-based interpretation without replacing the deterministic checker.

Important methods:

- UsedCarApplicationFacade.checkMissingInformation(vehicle : Vehicle) : MissingInformationResult
- DataQualityChecker.check(vehicle : Vehicle) : MissingInformationResult
- DataQualityChecker.checkConflict(manualVehicle : Vehicle, extractedVehicle : Vehicle) : MissingInformationResult
- LLMClient.generate(prompt : String) : String (only when ambiguity requires AI assistance)
- GeminiAdapter.generate(prompt : String) : String (only when ambiguity requires AI assistance)

Execution:

UsedCarApplicationFacade.checkMissingInformation(vehicle) sends the structured Vehicle to DataQualityChecker.check(vehicle). The checker deterministically evaluates required fields, explicit unknown values, and whether sufficient information exists to continue. When a manually entered Vehicle and an AI-extracted Vehicle disagree, checkConflict(manualVehicle, extractedVehicle) records the conflict instead of

silently choosing one value. The output is a MissingInformationResult that later features can inspect before making cost, risk, comparison, checklist, or recommendation decisions. SD02 shows this deterministic core before cost and constraint processing. If a seller statement is linguistically ambiguous, UsedCarAgent may use LLMClient/GeminiAdapter only to interpret that wording; the application still returns a structured MissingInformationResult.

Alternative / error behavior: If too much required information is missing, sufficient is false and downstream risk/recommendation features must avoid an overconfident result. Conflicting values remain unresolved until the user chooses which value to keep. Gemini interpretation is optional and never substitutes invented facts for absent data.

F05 — Local Vehicle Knowledge Retrieval

Related Use Case: UC05 — Retrieve Local Vehicle Knowledge

Related Sequence Diagram: SD03 — Knowledge Retrieval and AI-Assisted Risk Assessment

Classes involved:

- **UsedCarApplicationFacade** — exposes retrieveKnowledge(vehicle) to the presentation layer.
- **UsedCarAgent** — coordinates retrieval as part of a larger evaluation workflow.
- **ToolManager** — selects and executes registered tools through one common entry point.
- **VehicleKnowledgeTool** — adapts the generic tool call into a vehicle-knowledge lookup.
- **VehicleKnowledgeRepository** — loads and queries the local JSON/CSV knowledge base by normalized make, model, and year.
- **VehicleKnowledge** — stores the retrieved reliability summary, common issues, inspection notes, and source label.

Important methods:

- UsedCarApplicationFacade.retrieveKnowledge(vehicle : Vehicle) : VehicleKnowledge
- ToolManager.execute(name : String, input : Object) : Object
- VehicleKnowledgeTool.execute(input : Object) : Object
- VehicleKnowledgeRepository.load() : void
- VehicleKnowledgeRepository.findByVehicle(make : String, model : String, year : int) : VehicleKnowledge

Execution:

When knowledge is requested directly or as part of an evaluation, UsedCarAgent asks ToolManager.execute(name, input) to invoke VehicleKnowledgeTool. The tool extracts the normalized vehicle identity and queries VehicleKnowledgeRepository.findByVehicle(make, model, year). The repository searches the small local JSON/CSV data source and returns a VehicleKnowledge object containing only stored reliability/inspection evidence. VehicleKnowledgeTool returns this result through ToolManager to the agent and/or facade. SD03 shows the agent-to-tool-to-repository collaboration before AI risk synthesis. This separation makes the provenance of model-specific facts explicit: Gemini receives retrieved evidence only after deterministic local lookup has completed.

Alternative / error behavior: If the repository has no matching entry, the tool returns an explicit no-local-data result rather than fabricating model-specific knowledge. A repository/tool failure is reported as unavailable evidence so later AI output can be reduced in confidence or omitted.

F06 — Ownership Cost Estimation

Related Use Case: UC06 — Estimate Ownership Cost

Related Sequence Diagram: SD02 — Prepare Vehicle Evaluation

Classes involved:

• **UsedCarApplicationFacade** — accepts the vehicle and user-provided annual-distance/fuel-price assumptions.

• **ToolManager** — dispatches the calculation through the common Tool interface.

• **CostCalculatorTool** — performs deterministic ownership-cost calculations.

• **CostEstimate** — stores annual fuel use, annual fuel cost, and optional purchase-tax/first-year values when inputs are available.

• **Vehicle** — provides structured fuel-consumption and vehicle data used by the calculation.

Important methods:

• UsedCarApplicationFacade.estimateCost(vehicle : Vehicle, annualKm : double, fuelPrice : double) : CostEstimate

• ToolManager.execute(name : String, input : Object) : Object

• CostCalculatorTool.execute(input : Object) : Object

• CostCalculatorTool.calculateAnnualFuelCost(annualKm : double, fuelConsumption : double, fuelPrice : double) : double

Execution:

UsedCarApplicationFacade.estimateCost(vehicle, annualKm, fuelPrice) receives explicit assumptions from the user and passes a structured input to ToolManager. ToolManager invokes CostCalculatorTool.execute(input), which uses calculateAnnualFuelCost(annualKm, fuelConsumption, fuelPrice) and ordinary Java arithmetic to calculate the available cost metrics. The result is returned as CostEstimate and displayed or reused by comparison/risk/report features. SD02 shows this deterministic branch alongside missing-information and constraint preparation. Because no LLM participates in the calculation, the formula is repeatable, explainable, and suitable for direct unit testing in Stage 3.

Alternative / error behavior: Malformed or negative numeric input is rejected before the tool runs. If a required optional assumption or fuel-consumption value is unavailable, the corresponding estimate remains unavailable instead of being guessed by the program or Gemini.

F07 — Buyer Constraint Conflict Detection

Related Use Case: UC07 — Check Buyer Constraint Conflicts

Related Sequence Diagram: SD02 — Prepare Vehicle Evaluation

Classes involved:

• **UsedCarApplicationFacade** — provides checkConstraints(vehicle) and coordinates access to the stored buyer profile.

• **EvaluationSession** — returns the current UserPreference through getPreference().

• **ToolManager** — invokes the constraint tool through the common Tool abstraction.

• **ConstraintCheckerTool** — compares structured Vehicle data against explicitly stated UserPreference rules.

• **UserPreference** — contains the buyer's maximum budget, optional mileage limit, accepted transmissions, and other explicit constraints.

• **ConstraintViolation** — represents each failed rule with expected, actual, and severity information.

Important methods:

• UsedCarApplicationFacade.checkConstraints(vehicle : Vehicle) : List<ConstraintViolation>

• EvaluationSession.getPreference() : UserPreference

• ToolManager.execute(name : String, input : Object) : Object

• ConstraintCheckerTool.execute(input : Object) : Object

• ConstraintCheckerTool.check(vehicle : Vehicle, preference : UserPreference) : List<ConstraintViolation>

Execution:

UsedCarApplicationFacade.checkConstraints(vehicle) obtains the current buyer profile from EvaluationSession.getPreference(). The facade/tool layer packages the Vehicle and UserPreference and asks ToolManager to execute ConstraintCheckerTool. Through execute(input), the tool applies check(vehicle, preference) to only the constraints that the user has explicitly set. Each actual conflict becomes a structured ConstraintViolation. SD02 shows the violations returning through ToolManager and the facade to the GUI and becoming available to later agent workflows. Keeping this check deterministic ensures that Gemini can explain a conflict but cannot override the underlying rule result.

Alternative / error behavior: If a vehicle property needed for a constraint is unknown, that rule is marked unknown/not evaluable rather than being treated as passed. Unspecified buyer preferences do not create violations.

F08 — AI-Assisted Vehicle Risk Assessment

Related Use Case: UC08 — Evaluate Vehicle Risk

Related Sequence Diagram: SD03 — Knowledge Retrieval and AI-Assisted Risk Assessment

Classes involved:

• **EvaluateVehicleCommand** — encapsulates the evaluation request and lets GUI/CLI invoke the same high-level operation.

• **UsedCarApplicationFacade** — receives evaluateVehicle(vehicle) and delegates the multi-step workflow.

• **UsedCarAgent** — coordinates validation, retrieval, constraints, strategy selection, prompt construction, LLM use, and parsing.

• **DataQualityChecker** — supplies the current MissingInformationResult before AI synthesis.

• **ToolManager / VehicleKnowledgeTool / ConstraintCheckerTool** — provide deterministic local knowledge and constraint evidence.

• **EvaluationStrategyFactory** — creates the buyer-appropriate EvaluationStrategy.

• **EvaluationStrategy** — produces priority guidance without duplicating the overall workflow.

• **PromptBuilder** — builds the evidence-grounded risk prompt.

• **LLMClient / GeminiAdapter** — performs the provider-independent / provider-specific LLM interaction.

• **ResponseParser** — validates the LLM output and constructs RiskAssessment.

• **RiskAssessment** — stores riskLevel, strengths, risks, uncertainties, and explanation.

Important methods:

• EvaluateVehicleCommand.execute() : Object

• UsedCarApplicationFacade.evaluateVehicle(vehicle : Vehicle) : RiskAssessment

• UsedCarAgent.assessRisk(vehicle : Vehicle, preference : UserPreference) : RiskAssessment

• DataQualityChecker.check(vehicle : Vehicle) : MissingInformationResult

• ToolManager.execute(name : String, input : Object) : Object

• EvaluationStrategyFactory.createStrategy(preference : UserPreference) : EvaluationStrategy

• EvaluationStrategy.buildPriorityGuidance(preference : UserPreference) : String

• PromptBuilder.buildRiskPrompt(vehicle, knowledge, missing, violations) : String

• LLMClient.generate(prompt : String) : String

• GeminiAdapter.generate(prompt : String) : String

• ResponseParser.parseRiskAssessment(response : String) : RiskAssessment

Execution:

The user starts evaluation through EvaluateVehicleCommand.execute(), which invokes UsedCarApplicationFacade.evaluateVehicle(vehicle). UsedCarAgent.assessRisk(vehicle, preference) first obtains deterministic evidence: data-quality status, local VehicleKnowledge, and current constraint results. EvaluationStrategyFactory.createStrategy(preference) returns BalancedStrategy, BudgetFocusedStrategy, or ReliabilityFocusedStrategy through the EvaluationStrategy interface. buildPriorityGuidance(preference) adds the buyer-specific emphasis without changing the surrounding workflow. PromptBuilder.buildRiskPrompt(...) packages the structured evidence, and the agent invokes LLMClient.generate(prompt enriched with priority guidance); GeminiAdapter handles the actual Gemini API interaction. ResponseParser.parseRiskAssessment(response) validates the model output and creates RiskAssessment, which is returned through the facade to the GUI. SD03 traces this complete agent/tool/strategy/LLM collaboration.

Alternative / error behavior: If MissingInformationResult is insufficient, the system can stop synthesis and return Insufficient Information / trigger clarification. Missing local knowledge is represented explicitly. If Gemini fails or its output cannot be parsed, deterministic evidence remains available and the AI risk narrative is marked unavailable rather than fabricated.

F09 — Two-Vehicle Comparison

Related Use Case: UC09 — Compare Two Vehicles

Related Sequence Diagram: SD04 — Two-Vehicle Comparison

Classes involved:

• **CompareVehiclesCommand** — encapsulates the compare request for GUI/CLI use.

• **UsedCarApplicationFacade** — provides compareVehicles(first, second) as the application-level operation.

• **UsedCarAgent** — coordinates symmetrical evidence gathering and final comparison synthesis.

• **ToolManager** — runs the same deterministic tools for each candidate.

• **CostCalculatorTool** — produces comparable cost evidence.

• **ConstraintCheckerTool** — applies the same buyer constraints to each vehicle.

• **VehicleKnowledgeTool** — retrieves local reliability/inspection evidence for each vehicle.

• **PromptBuilder** — constructs the comparison prompt.

• **LLMClient / GeminiAdapter** — generates the trade-off explanation using structured evidence.

• **ResponseParser** — converts the model response into VehicleComparison.

• **VehicleComparison** — stores metricSummary, vehicle identifiers, preferredVehicleId when supportable, and the explanation.

Important methods:

• CompareVehiclesCommand.execute() : Object

• UsedCarApplicationFacade.compareVehicles(first : Vehicle, second : Vehicle) : VehicleComparison

• UsedCarAgent.compare(first : Vehicle, second : Vehicle, preference : UserPreference) : VehicleComparison

• ToolManager.execute(name : String, input : Object) : Object

• CostCalculatorTool.execute(input : Object) : Object

• ConstraintCheckerTool.execute(input : Object) : Object

• VehicleKnowledgeTool.execute(input : Object) : Object

• PromptBuilder.buildComparisonPrompt(first : Vehicle, second : Vehicle) : String

• LLMClient.generate(prompt : String) : String

• GeminiAdapter.generate(prompt : String) : String

• ResponseParser.parseComparison(response : String) : VehicleComparison

Execution:

CompareVehiclesCommand.execute() invokes UsedCarApplicationFacade.compareVehicles(first, second), which delegates to UsedCarAgent.compare(first, second, preference). In the SD04 loop, ToolManager applies the same cost, constraint, and vehicle-knowledge operations to Vehicle A and Vehicle B so both candidates are evaluated using the same criteria. The agent builds a structured side-by-side evidence context. PromptBuilder.buildComparisonPrompt(first, second) produces the base comparison request, and LLMClient.generate(prompt enriched with structured evidence) is serviced by GeminiAdapter. ResponseParser.parseComparison(response) validates the model output and returns a VehicleComparison. The GUI can therefore display fixed metrics separately from the AI-generated trade-off explanation.

Alternative / error behavior: If a metric or local knowledge item is missing for one vehicle, it is shown as unavailable instead of being guessed. If only one vehicle is selected, comparison is rejected. If Gemini is unavailable, the deterministic side-by-side metrics remain usable while the narrative trade-off explanation is marked unavailable.

F10 — Vehicle-Specific Inspection Checklist Generation

Related Use Case: UC10 — Generate Vehicle-Specific Inspection Checklist

Related Sequence Diagram: SD05 — Inspection Checklist and Clarification Loop

Classes involved:

• **GenerateChecklistCommand** — encapsulates checklist generation for reuse by GUI/CLI.

• **UsedCarApplicationFacade** — provides generateChecklist(vehicle) as the high-level entry point.

• **UsedCarAgent** — coordinates data quality, local knowledge, prompt creation, LLM use, and parsing.

• **DataQualityChecker** — supplies missing/uncertain information that should influence inspection priorities.

• **ToolManager / VehicleKnowledgeTool** — retrieve model-specific local reliability and inspection evidence when available.

• **PromptBuilder** — constructs a constrained checklist prompt from Vehicle, VehicleKnowledge, and MissingInformationResult.

• **LLMClient / GeminiAdapter** — generates the checklist text through the provider abstraction.

• **ResponseParser** — validates and converts the response into InspectionChecklist.

• **InspectionChecklist** — stores grouped inspection sections and seller questions.

Important methods:

• GenerateChecklistCommand.execute() : Object

• UsedCarApplicationFacade.generateChecklist(vehicle : Vehicle) : InspectionChecklist

• UsedCarAgent.generateChecklist(vehicle : Vehicle) : InspectionChecklist

• DataQualityChecker.check(vehicle : Vehicle) : MissingInformationResult

• ToolManager.execute(name : String, input : Object) : Object

• VehicleKnowledgeTool.execute(input : Object) : Object

• PromptBuilder.buildChecklistPrompt(vehicle : Vehicle, knowledge : VehicleKnowledge, missing : MissingInformationResult) : String

• LLMClient.generate(prompt : String) : String

• GeminiAdapter.generate(prompt : String) : String

• ResponseParser.parseChecklist(response : String) : InspectionChecklist

Execution:

The user requests a checklist through GenerateChecklistCommand.execute(), which invokes UsedCarApplicationFacade.generateChecklist(vehicle). UsedCarAgent.generateChecklist(vehicle) retrieves the current MissingInformationResult and asks ToolManager/VehicleKnowledgeTool for model-specific evidence. PromptBuilder.buildChecklistPrompt(vehicle, knowledge, missing) turns those structured facts into a constrained request. LLMClient.generate(prompt) is implemented at runtime by GeminiAdapter, and ResponseParser.parseChecklist(response) converts the result into InspectionChecklist before it is displayed. SD05 shows the tool/LLM path and the relationship between checklist generation and subsequent clarification. The design keeps provider-specific API code out of the GUI and keeps local knowledge distinct from generated wording.

Alternative / error behavior: If no local vehicle knowledge exists, the system produces a clearly generic/reduced-confidence checklist rather than inventing model-specific issues. If Gemini is unavailable, deterministic missing-information prompts and a basic generic inspection fallback can still be presented where possible.

F11 — Clarification Question and Follow-Up Management

Related Use Case: UC11 — Provide Clarification and Resume Evaluation

Related Sequence Diagram: SD05 — Inspection Checklist and Clarification Loop

Classes involved:

• **UsedCarAgent** — generates a small prioritized clarification set when MissingInformationResult shows that additional information would materially improve evaluation.

• **PromptBuilder** — constructs the clarification prompt from the structured MissingInformationResult.

• **LLMClient / GeminiAdapter** — generates clarification wording while hiding provider-specific API details.

• **ResponseParser** — validates that the output is a small list of usable clarification questions.

• **UsedCarApplicationFacade** — accepts each user answer through submitClarification(field, value) and applies deterministic validation/update logic.

• **EvaluationSession** — records clarification history and stores the updated Vehicle as persistent session context.

• **MissingInformationResult** — identifies which fields are missing/uncertain and motivates clarification.

• **Vehicle** — is updated only with user-provided/resolved information; skipped values remain unknown.

Important methods:

• UsedCarAgent.generateClarificationQuestions(result : MissingInformationResult) : List<String>

• PromptBuilder.buildClarificationPrompt(missing : MissingInformationResult) : String

• LLMClient.generate(prompt : String) : String

• GeminiAdapter.generate(prompt : String) : String

• ResponseParser.parseClarificationQuestions(response : String) : List<String>

• UsedCarApplicationFacade.submitClarification(field : String, value : String) : void

• EvaluationSession.recordClarification(text : String) : void

• EvaluationSession.setCurrentVehicle(vehicle : Vehicle) : void

• Vehicle.updateField(name : String, value : Object) : void

Execution:

When clarification would improve the evaluation, UsedCarAgent.generateClarificationQuestions(result) asks PromptBuilder.buildClarificationPrompt(missing) for a focused request. Gemini is called only through LLMClient/GeminiAdapter, and ResponseParser.parseClarificationQuestions(response) validates one to three prioritized questions. The list is generated once. SD05 then loops over the already-generated questions (maximum three): the user answers, skips, or marks a value unknown; UsedCarApplicationFacade.submitClarification(field, value) validates the response; the Vehicle is updated when appropriate; EvaluationSession.recordClarification(text) preserves lightweight interaction history; and setCurrentVehicle(updatedVehicle) stores the new structured state. Later evaluation can resume using this updated context.

Alternative / error behavior: A skipped/unknown answer remains unknown instead of being inferred by the model. If a clarification conflicts with existing data, the GUI displays both values and the user explicitly chooses which value to retain. If Gemini is unavailable or returns malformed questions, the system avoids fabricated answers and can fall back to deterministic missing-field prompts.

F12 — Personalized Purchase Recommendation and Evaluation Report

Related Use Case: UC12 — Generate Personalized Recommendation Report

Related Sequence Diagram: SD06 — Personalized Recommendation and Final Evaluation Report

Classes involved:

- **UsedCarApplicationFacade** — provides generateReport(vehicle) and obtains the stored UserPreference from EvaluationSession.
- **EvaluationSession** — supplies UserPreference and retains current vehicle/clarification context as session state.
- **UsedCarAgent** — combines previously produced structured evidence and coordinates final recommendation synthesis.
- **EvaluationStrategyFactory** — creates the strategy that matches the buyer's priorities.
- **EvaluationStrategy** — produces priorityGuidance used to personalize the final synthesis.
- **PromptBuilder** — constructs an evidence-grounded final recommendation prompt.
- **LLMClient / GeminiAdapter** — generates the advisory narrative through the provider abstraction.
- **ResponseParser** — validates the response and constructs EvaluationReport.
- **EvaluationReport** — stores the final category, strengths, risks, missing information, cost summary, inspection priorities, next steps, and explanation.
- **RiskAssessment / CostEstimate / ConstraintViolation / VehicleKnowledge / MissingInformationResult** — represent previously generated deterministic/hybrid evidence consumed by the final report workflow; SD06 reuses this evidence rather than recalculating it.

Important methods:

- UsedCarApplicationFacade.generateReport(vehicle : Vehicle) : EvaluationReport
- EvaluationSession.getPreference() : UserPreference
- UsedCarAgent.generateRecommendation(vehicle : Vehicle, preference : UserPreference) : EvaluationReport
- EvaluationStrategyFactory.createStrategy(preference : UserPreference) : EvaluationStrategy
- EvaluationStrategy.buildPriorityGuidance(preference : UserPreference) : String
- PromptBuilder.buildRecommendationPrompt(vehicle : Vehicle, risk : RiskAssessment, cost : CostEstimate, violations : List<ConstraintViolation>) : String
- LLMClient.generate(prompt : String) : String

• GeminiAdapter.generate(prompt : String) : String

• ResponseParser.parseReport(response : String) : EvaluationReport

Execution:

The user selects Generate Recommendation / Report and UsedCarApplicationFacade.generateReport(vehicle) obtains the current UserPreference through EvaluationSession.getPreference(). The vehicle itself is supplied to generateReport, while clarification/current-state context remains available as session state; no undeclared getter is required. UsedCarAgent.generateRecommendation(vehicle, preference) combines the latest RiskAssessment with available cost, constraint, local-knowledge, missing-information, and clarification evidence produced by earlier workflows. EvaluationStrategyFactory.createStrategy(preference) selects the appropriate EvaluationStrategy and buildPriorityGuidance(preference) supplies buyer-specific emphasis. PromptBuilder.buildRecommendationPrompt(...) creates the base final prompt, and LLMClient.generate(prompt enriched with priorityGuidance and structured evidence) is fulfilled by GeminiAdapter. ResponseParser.parseReport(response) constructs EvaluationReport, which the facade returns to the GUI. SD06 shows the final advisory categories: Strong Candidate, Consider with Caution, High Risk, or Insufficient Information.

Alternative / error behavior: If evidence is insufficient, the category is Insufficient Information rather than a confident recommendation. If one evidence source is unavailable, the report identifies the missing evidence and continues only with what is available. If Gemini is unavailable or its output remains malformed after a controlled retry, deterministic evidence remains viewable while the AI narrative is marked unavailable. The result is advisory: no purchase outcome is guaranteed.

9.1 Consistency Check

Each feature above has an explicit path from user interaction to use case, classes, methods, runtime sequence, structured result, and error handling. The same application design is reused across GUI and CLI through the facade/command boundaries. Gemini is isolated behind LLMClient/GeminiAdapter, while validation, retrieval, calculations, constraint enforcement, state management, and response parsing remain explicit software components. This preserves the Stage 1 traceability goal and gives Stage 2 implementation and Stage 3 testing concrete targets.